

user-3 EC2-Admin View, Start, and Stop Amazon EC2 instances
Tasks:

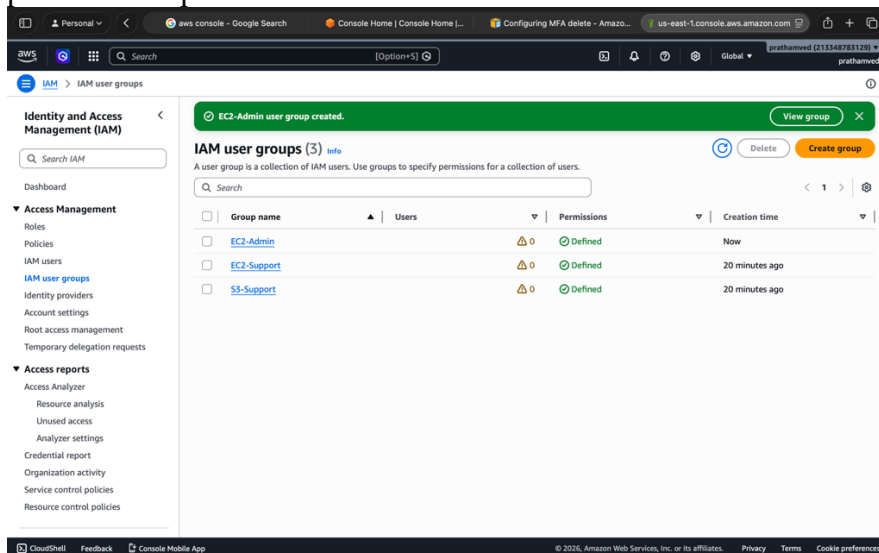
Step 1: Create IAM Groups and Attach Policies

In the AWS Management Console, navigate to **IAM > Groups > Create New Group**.

1. Create the required IAM groups: S3-Support, EC2-Support, and EC2-Admin

Group Name	Managed Policy to Attach
S3-Support	AmazonS3ReadOnlyAccess
EC2-Support	AmazonEC2ReadOnlyAccess
EC2-Admin	AmazonEC2FullAccess

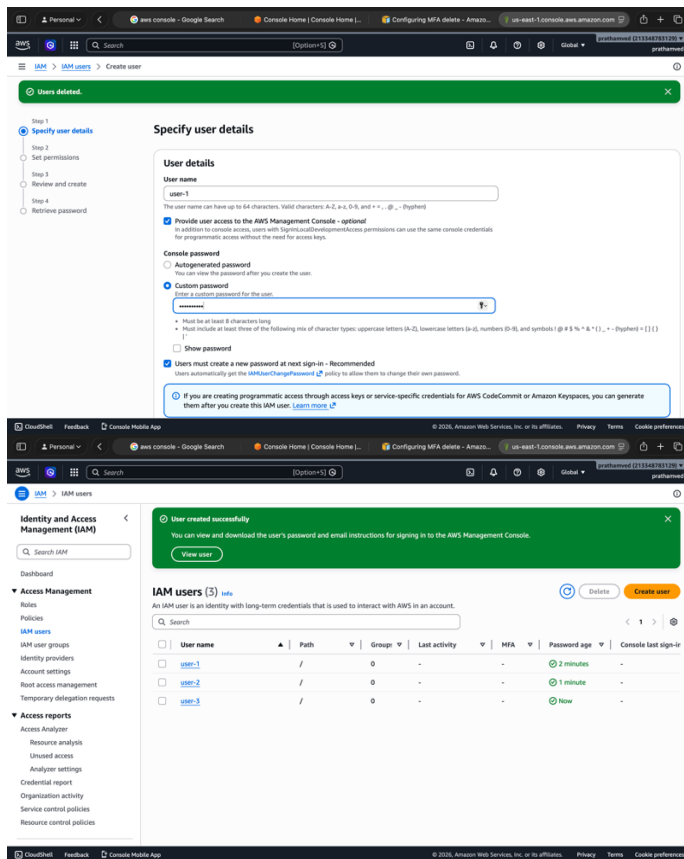
2. Attach appropriate managed policies to each group based on the permissions specified.



3. Create IAM users: user-1, user-2, and user-3

Navigate to **IAM > Users > Create user**.

1. Create **user-1**, **user-2**, and **user-3**.
2. Select **AWS Management Console access**.
3. Set a custom password or use an autogenerated one.
4. **Do not** add them to groups yet (we will do this in Step 5).



4. Implement Multi-Factor Authentication (MFA) for all users and enforce it using an IAM policy.

Create the MFA Enforcement Policy

By default, "enforcing" MFA requires a custom JSON policy that denies actions if the user is not signed in with MFA.

1. Go to **IAM > Policies > Create Policy**.
2. Click the **JSON** tab and paste a policy that uses the `aws:MultiFactorAuthPresent` condition.
3. Name it **MFA-Enforcement-Policy**.

Note: A standard enforcement policy effectively says: *"Allow these actions only IF `aws:MultiFactorAuthPresent` is true; otherwise, Deny."*

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowUserToManageTheirOwnMFA",
```

```
"Effect": "Allow",

"Action": [

    "iam:CreateVirtualMFADevice",

    "iam:EnableMFADevice",

    "iam:ListVirtualMFADevices",

    "iam:ResyncMFADevice"

],

"Resource": [

    "arn:aws:iam::*:mfa/${aws:username}",

    "arn:aws:iam::*:user/${aws:username}"

]

},

{

    "Sid": "DenyEverythingIfNoMFA",

    "Effect": "Deny",

    "NotAction": [

        "iam:CreateVirtualMFADevice",

        "iam:EnableMFADevice",

        "iam:ListVirtualMFADevices",

        "sts:GetSessionToken"

    ],

    "Resource": "*",

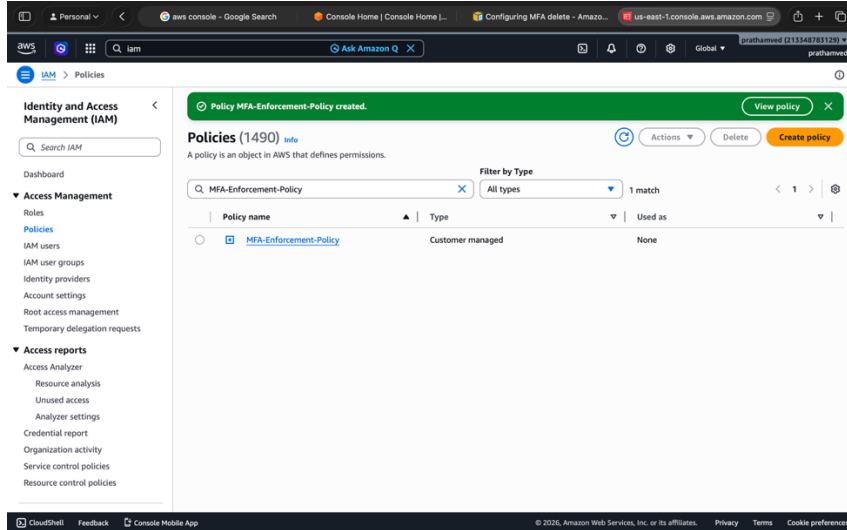
    "Condition": {

        "BoolIfExists": { "aws:MultiFactorAuthPresent": "false" }

    }

}
```

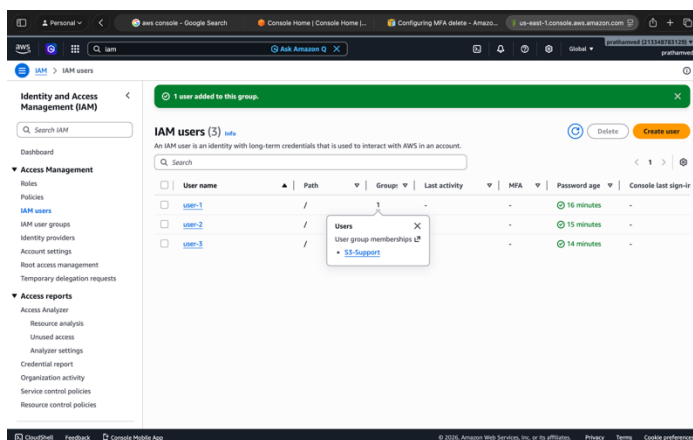
```
}
]
}
```

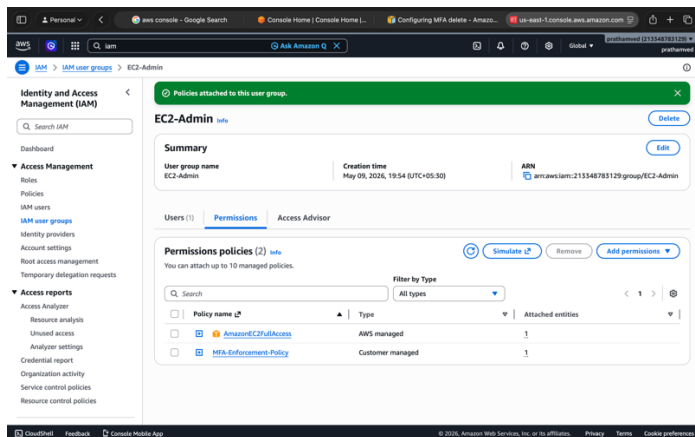


5. Assign each user to the corresponding group.

Navigate to **IAM > Groups** and add the users to their respective buckets:

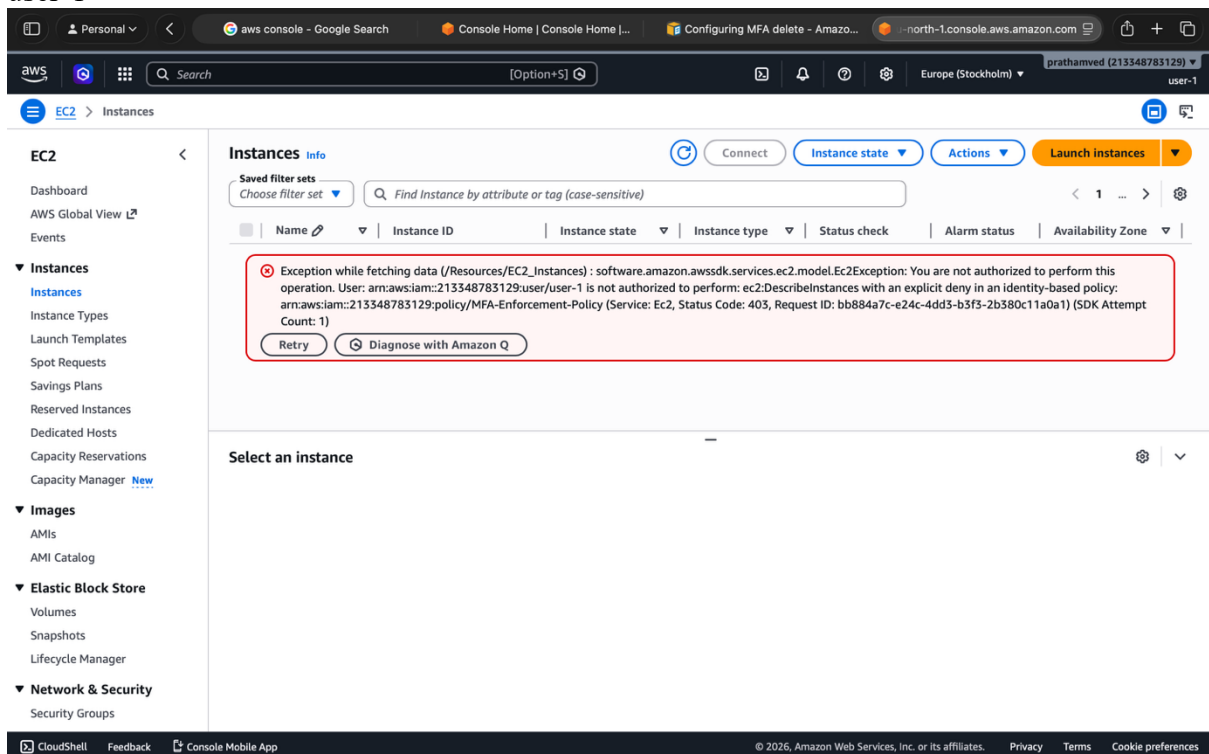
- **S3-Support:** Add user-1
- **EC2-Support:** Add user-2
- **EC2-Admin:** Add user-3
- **All Groups:** Attach the **MFA-Enforcement-Policy** (created in Step 3) to every group or directly to the users.





All Groups: Attach the **MFA-Enforcement-Policy** (created in Step 3) to every group or directly to the users

6. Verify that permissions are correctly applied by testing access levels.
user-1



User	Expected Result
user-1	Can view S3 buckets; cannot start/stop EC2.
user-2	Can view EC2 instances; cannot start/stop them.
user-3	Can View, Start, and Stop EC2 instances.
All Users	If logged in without MFA, they should be denied access to everything (due to the enforcement policy).

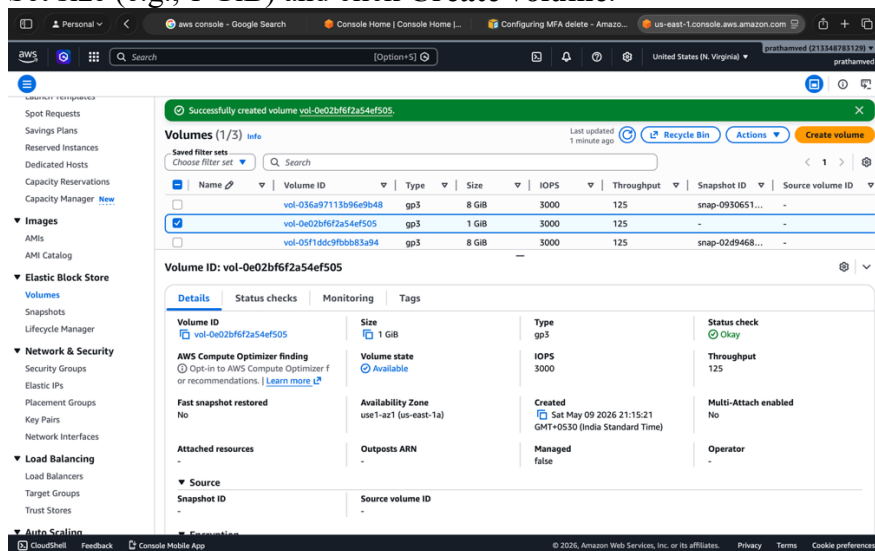
Q2. Using Amazon Web Services, demonstrate storage provisioning, backup, and data recovery using Amazon EBS with Amazon EC2.

This is a classic disaster recovery lab for your practicals. Since you are already working with EC2-Admin (user-3), you have the full permissions needed to handle EBS volumes and snapshots.

Follow these steps to complete the storage and backup lifecycle:

Step 1: Create an Amazon EBS Volume

1. Go to the **EC2 Dashboard**.
2. In the left menu, under **Elastic Block Store**, select **Volumes**.
3. Click **Create volume**.
4. **Important:** Ensure the **Availability Zone** matches exactly where your EC2 instance is running (e.g., us-east-1a).
5. Set size (e.g., 1 GiB) and click **Create volume**.



Step 2: Attach to EC2 Instance

1. Select your new volume, click **Actions**, and choose **Attach volume**.
2. Select your running **EC2 instance**.

Note the device name (usually `/dev/sdf` or `/dev/xvdf`). Click **Attach**.

The screenshot shows the AWS Management Console interface for the 'Attach volume' page. The breadcrumb navigation is 'EC2 > Volumes > vol-050e814dc5d7efb20 > Attach volume'. The page title is 'Attach volume' with an 'Info' link. Below the title is a description: 'Attach a volume to an instance to use it as you would a regular physical hard disk drive.' The 'Basic details' section contains the following information: 'Volume ID' is 'vol-050e814dc5d7efb20 (ccl)'; 'Availability Zone' is 'use1-az1 (us-east-1a)'; 'Instance' is a dropdown menu showing 'i-06d9cb787f58b3a55 (CCL-Lab-Instance) (running)'; 'Device name' is a dropdown menu showing '/dev/sdf'. A note states: 'Recommended device names for Linux: /dev/xvda for root volume, /dev/sd[f-p] for data volumes.' A warning message at the bottom of the details section says: 'Newer Linux kernels may rename your devices to /dev/xvdf through /dev/xvdp internally, even when the device name entered here (and shown in the details) is /dev/sdf through /dev/sdp.' At the bottom right of the details section are 'Cancel' and 'Attach volume' buttons.

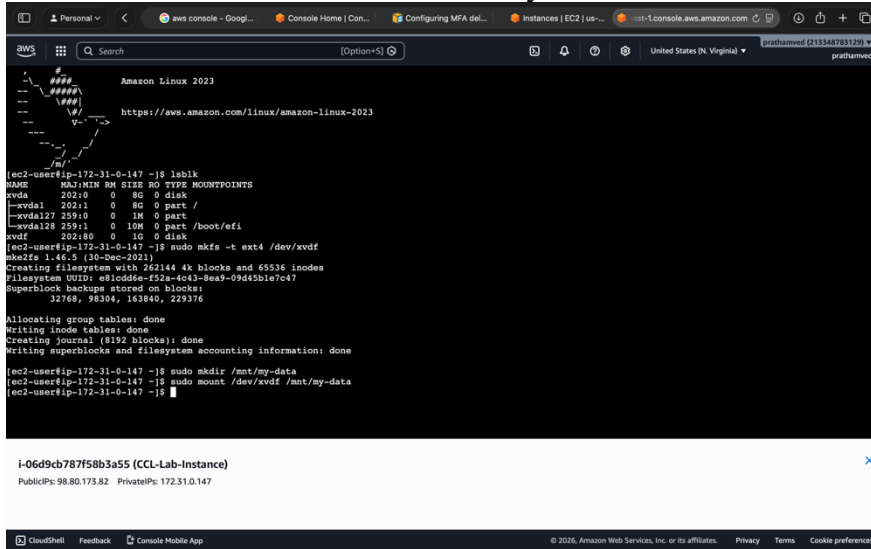
Step 3: Configure and Mount the File System

Connect to your instance via SSH or EC2 Instance Connect and run these commands:

The screenshot shows the AWS Management Console interface for the 'Connect' page. The breadcrumb navigation is 'EC2 > Instances > i-06d9cb787f58b3a55 > Connect to instance'. The page title is 'Connect' with an 'Info' link. Below the title is a description: 'Connect to an instance using the browser-based client.' The page displays instance details: 'Instance ID' is 'i-06d9cb787f58b3a55 (CCL-Lab-Instance)'; 'VPC ID' is 'vpc-0915d316a1422e2fe'; 'Security groups' is 'sg-04e551220ad12c376 (launch-wizard-3)'; 'IAM role' is '-'. Below this is a tabbed interface with 'EC2 Instance Connect' selected. The 'EC2 Instance Connect' section shows 'Instance ID' as 'i-06d9cb787f58b3a55 (CCL-Lab-Instance)'. Under 'Connection type', 'Connect using a Public IP' is selected. Below this, 'Public IPv4 address' is '98.80.173.82' and 'IPv6 address' is '-'. Under 'Username', 'ec2-user' is entered. A note at the bottom says: 'Note: In most cases, the default username, ec2-user, is correct. However, read your AMI usage instructions to check if the AMI owner has changed the default AMI username.' At the bottom right are 'Cancel' and 'Connect' buttons.

1. **Check the disk:** `lsblk` (You should see your 1GB disk listed).
2. **Create file system:** `sudo mkfs -t ext4 /dev/xvdf`
3. **Create mount point:** `sudo mkdir /mnt/my-data`

4. Mount it: `sudo mount /dev/xvdf /mnt/my-data`



```
Amazon Linux 2023
https://aws.amazon.com/linux/amazon-linux-2023

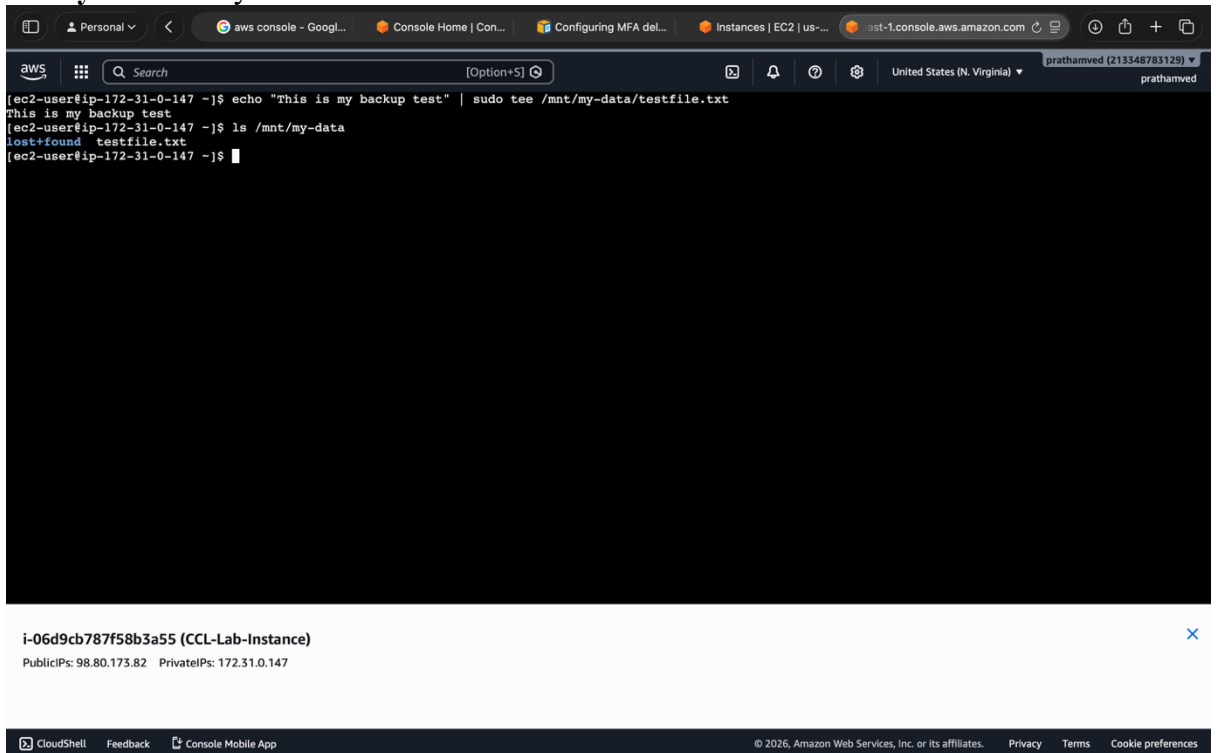
[ec2-user@ip-172-31-0-147 ~]$ lsblk
NAME        MAJ:MIN RM SIZE RO TYPE MOUNTPOINTS
xvda        202:10   0  8G  0 disk 
|xvda1      202:11   0   8G  0 part /
|xvda127    259:10   0  1M  0 part 
|xvda128    259:11   0  10M  0 part /boot/efi
xvdf        202:80   0   1G  0 disk 
[ec2-user@ip-172-31-0-147 ~]$ sudo mkfs -t ext4 /dev/xvdf
mkfs: 1.46.5 (30-Dec-2021)
Creating filesystem with 262144 4k blocks and 65536 inodes
Filesystem UUID: e81cdd6e-f52a-4c43-8ea9-09d45b1e7c47
Superblock backups stored on blocks:
32768, 98304, 163840, 229376

Allocating group tables: done
Writing inode tables: done
Creating journal (8192 blocks): done
Writing superblocks and filesystem accounting information: done

[ec2-user@ip-172-31-0-147 ~]$ sudo mkdir /mnt/my-data
[ec2-user@ip-172-31-0-147 ~]$ sudo mount /dev/xvdf /mnt/my-data
[ec2-user@ip-172-31-0-147 ~]$
```

Step 4: Store Sample Data

1. **Create a file:** `echo "This is my backup test" | sudo tee /mnt/my-data/testfile.txt`
2. **Verify:** `ls /mnt/my-data`



```
[ec2-user@ip-172-31-0-147 ~]$ echo "This is my backup test" | sudo tee /mnt/my-data/testfile.txt
This is my backup test
[ec2-user@ip-172-31-0-147 ~]$ ls /mnt/my-data
lost+found  testfile.txt
[ec2-user@ip-172-31-0-147 ~]$
```

Step 5: Create EBS Snapshot (Backup)

1. Go back to the **Volumes** screen in the AWS Console.
2. Select your volume -> **Actions** -> **Create snapshot**.
3. Add a description like "Lab Backup" and click **Create snapshot**.

4. Wait for the status in the **Snapshots** menu to show **Completed**.

The screenshot shows the 'Create snapshot' page in the AWS Management Console. The page is titled 'Create snapshot' and includes a sub-header 'Create a point-in-time snapshot to back up the data on an Amazon EBS volume to Amazon S3.' The 'Source volume' section shows 'Volume ID' as 'vol-050e814dc5d7efb20' and 'Availability Zone' as 'us-east-1a (us-east-1a)'. The 'Snapshot details' section has a 'Description' field with the value 'Lab Backup' and 'Encryption' set to 'Not encrypted'. The 'Tags' section shows 'No tags associated with the resource.' and an 'Add new tag' button. At the bottom right, there are 'Cancel' and 'Create snapshot' buttons.

Step 6: Restore Data (Create New Volume)

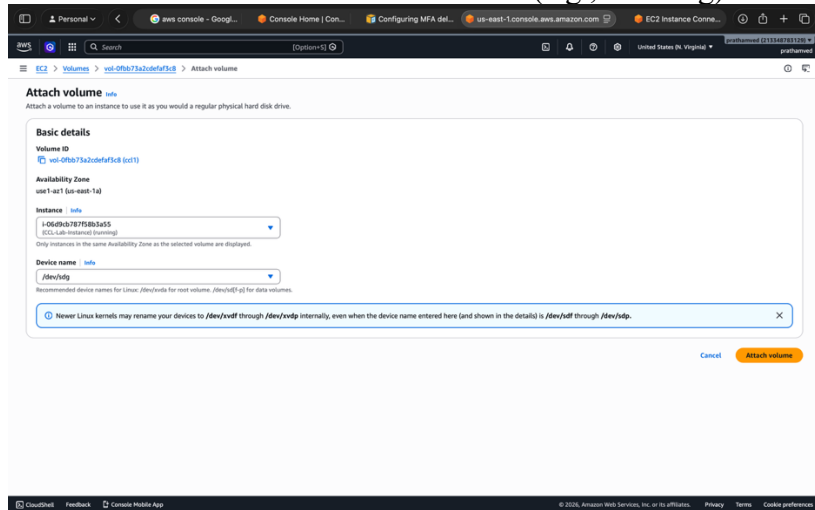
1. In the **Snapshots** menu, select your snapshot.
2. Click **Actions** -> **Create volume from snapshot**.
3. Again, ensure the **Availability Zone** matches your EC2 instance. Click **Create volume**

The screenshot shows the 'Create volume' page in the AWS Management Console. The page is titled 'Create volume' and includes a sub-header 'Create an Amazon EBS volume to attach to any EC2 instance in the same Availability Zone.' The 'Volume settings' section shows 'Snapshot ID' as 'snap-047535897732ed', 'Volume type' as 'General Purpose SSD (gp3)', 'Size (GB)' as '1', 'IOPS' as '3', 'Throughput (MB/s)' as '125', and 'Availability Zone' as 'us-east-1a (us-east-1a)'. The 'Fast snapshot restore' section is set to 'Not enabled for selected snapshot'. The 'Volume initialization rate (MB/s) - new, optional' section is set to 'Enter a value'. The 'Encryption' section is set to 'Not encrypted this volume'. The 'Tags' section shows 'No tags associated with the resource.' and an 'Add new tag' button. At the bottom right, there are 'Cancel' and 'Create volume' buttons.

Step 7: Attach Restored Volume

1. Go to **Volumes**, select the *new* volume you just created from the snapshot.
2. Click **Actions** -> **Attach volume** and pick the same EC2 instance.

- *Note:* It will use a different device name (e.g., /dev/sdg).

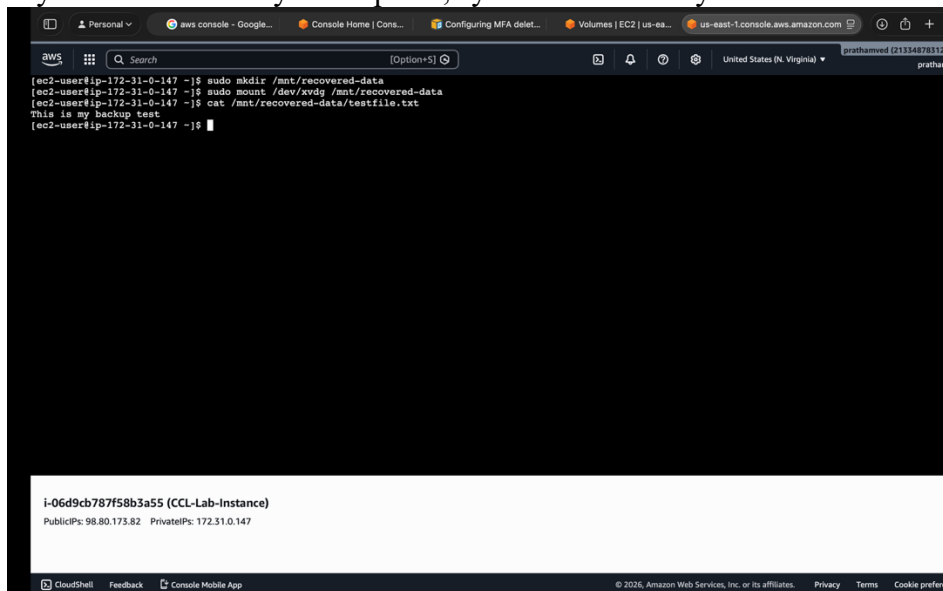


Step 8: Mount and Verify Recovery

Back in your terminal:

1. **Create a new mount point:** `sudo mkdir /mnt/recovered-data`
2. **Mount the restored disk:** `sudo mount /dev/xvdf /mnt/recovered-data`
 - *Note:* You don't need `mkfs` here because the data is already there!
3. **The Final Check:** `cat /mnt/recovered-data/testfile.txt`

If you see "This is my backup test," your data recovery is successful!



Q3 To Store and retrieve files using AWS S3

Task

Step 1: Open Amazon S3

1. Log in to the **AWS Management Console**.
2. In the search bar at the top, type **S3** and select it from the services list.

Step 2: Create a New S3 Bucket

1. Click the orange **Create bucket** button.
2. **Bucket name:** Enter a unique name (e.g., ccl-practical-yourname-2026). Bucket names must be globally unique across all of AWS.
3. **Region:** Keep the default (e.g., us-east-1).
4. **Object Ownership:** Keep "ACLs disabled" (recommended) or "ACLs enabled" if your lab specifically requires older access methods.
5. **Block Public Access settings:** For this specific lab (Task 4), you must **uncheck** the box that says "Block *all* public access."
 - *Note:* You will have to check a box acknowledging that this makes the bucket potentially public.
6. Click **Create bucket**.

Create bucket [info](#)

Buckets are containers for data stored in S3.

General configuration [info](#)

AWS Region
US East (N. Virginia) us-east-1

Bucket type [info](#)

☒ **General purpose**
Recommended for most use cases and access patterns. General purpose buckets are the original S3 bucket type. They allow a mix of storage classes that redundantly store objects across multiple Availability Zones.

☐ **Directory**
Recommended for low-latency use cases. These buckets use only the S3 Express One Zone storage class, which provides faster processing of data within a single Availability Zone.

Bucket namespace
Choose the namespace where you want to create your bucket. [Learn more](#) [info](#)

☒ **Global namespace**
By default, S3 creates general purpose buckets in the global namespace.

☐ **Account Regional namespace (recommended)**
General purpose buckets created in your account regional namespace are unique to your account. These buckets can never be created by another AWS account.

Bucket name [info](#)

ccl-practical-yourname-2026

Bucket names must be 3 to 63 characters and unique within the global namespace. Bucket names must also begin and end with a letter or number. Valid characters are a-z, 0-9, periods (.), and hyphens (-). [Learn more](#) [info](#)

Copy settings from existing bucket - optional
Only the bucket settings in the following configuration are copied.

[Choose bucket](#)

Format: s3://bucket/prefix

Object Ownership [info](#)

Control ownership of objects written to this bucket from other AWS accounts and the use of access control lists (ACLs). Object ownership determines who can specify access to objects.

Object Ownership

☒ **ACLs disabled (recommended)**
All objects in this bucket are owned by this account. Access to this bucket and its objects is specified using only policies.

☐ **ACLs enabled**
Objects in this bucket can be owned by other AWS accounts. Access to this bucket and its objects can be specified using ACLs.

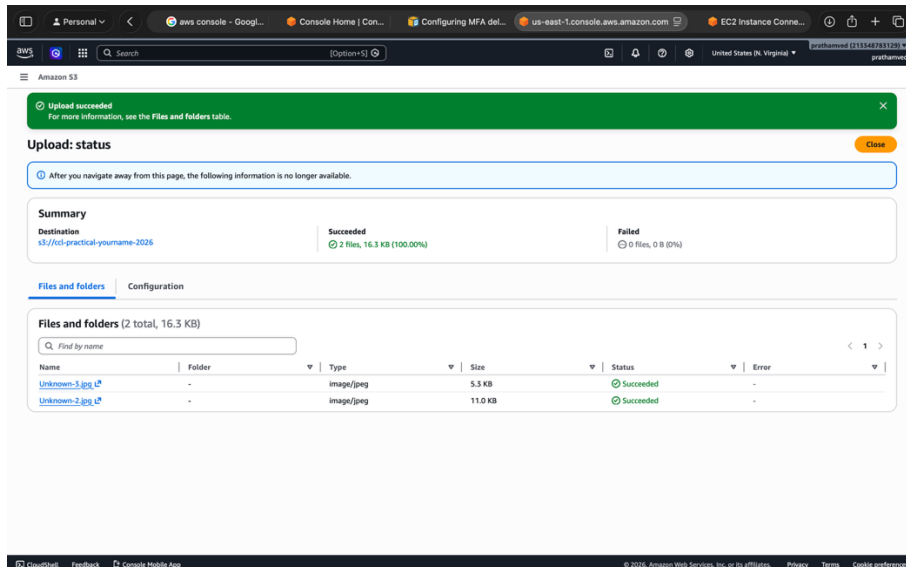
Object Ownership

[CloudShell](#) [Feedback](#) [Console Mobile App](#)

© 2026, Amazon Web Services, Inc. or its affiliates. [Privacy](#) [Terms](#) [Cookie preferences](#)

Step 3: Upload Files

1. Click on the name of your newly created bucket.
2. Click **Upload** -> **Add files**.
3. Select a text document and an image from your computer.
4. Click the orange **Upload** button at the bottom.



4. Configure public access permissions for uploaded files.

Step 4: Configure Public Access Permissions

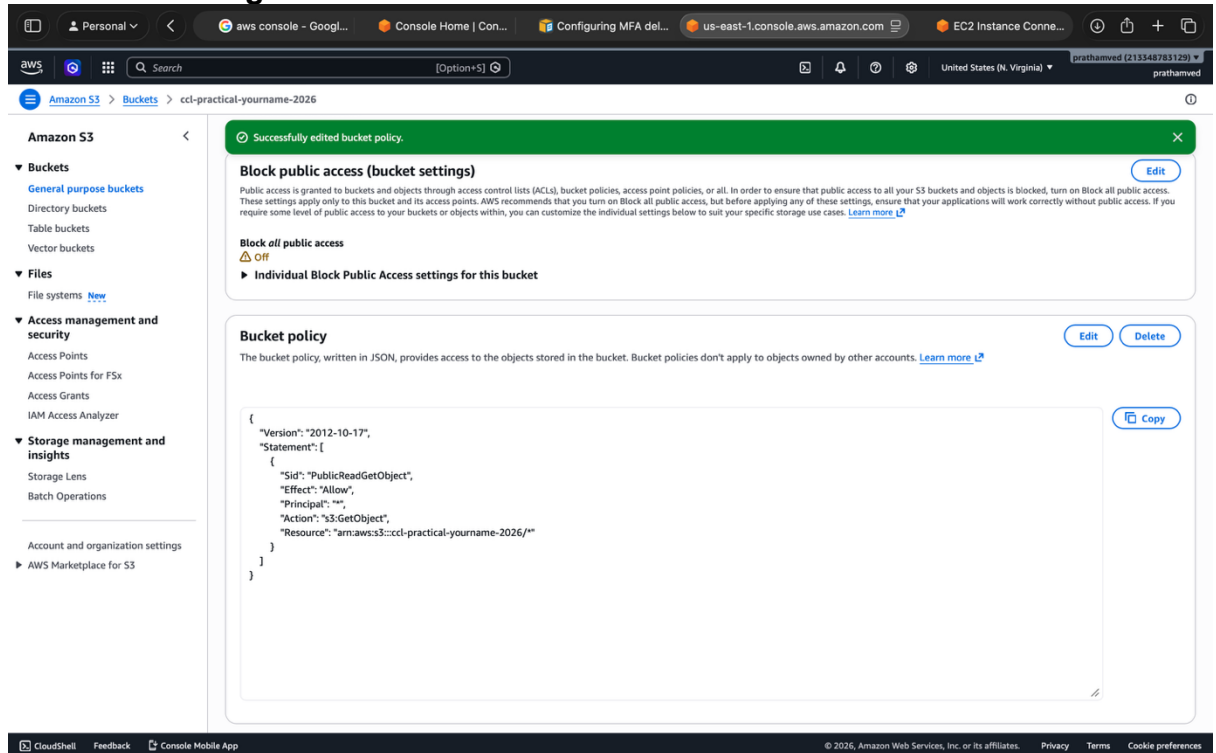
By default, even if the bucket allows public access, individual files are private. To make them viewable via URL:

1. **Bucket Policy (The easiest way):**
 - Go to the **Permissions** tab of your bucket.
 - Scroll down to **Bucket policy** and click **Edit**.
 - Paste this policy (replace YOUR-BUCKET-NAME with your actual bucket name):

JSON

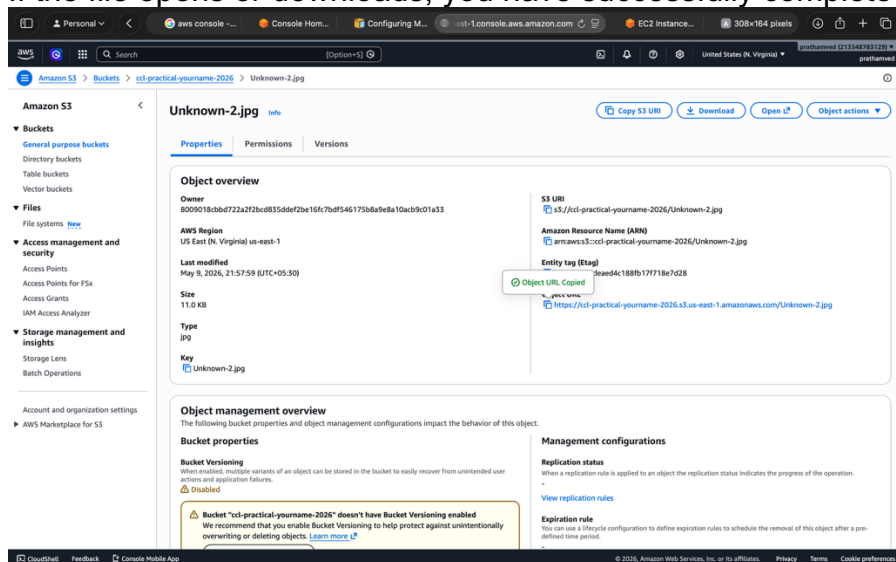
```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "PublicReadGetObject",
      "Effect": "Allow",
      "Principal": "*",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::ccl-practical-yourname-2026/*"
    }
  ]
}
```

2. Click **Save changes**.



Step 5: Access and Verify via URL

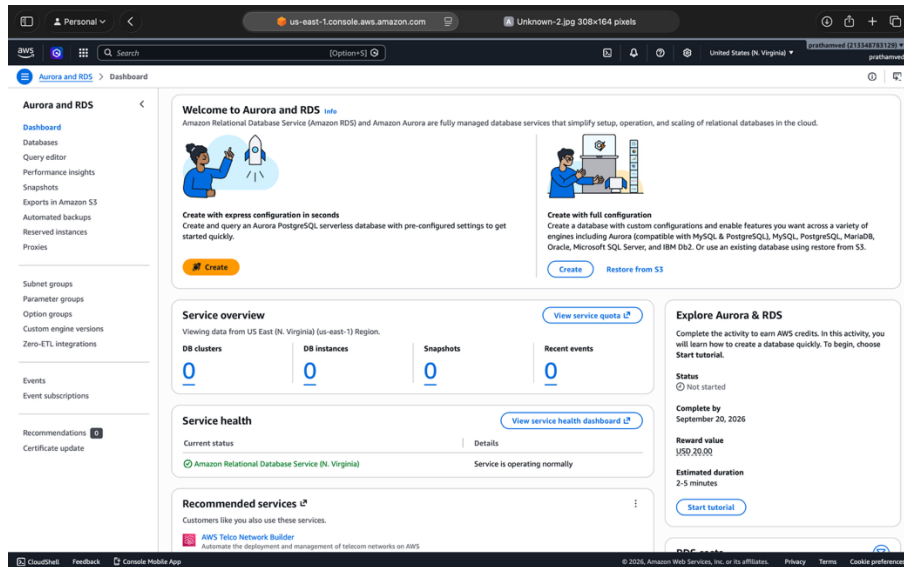
1. Go back to the **Objects** tab.
2. Click on the name of one of the files you uploaded (e.g., image.jpg).
3. Find the **Object URL** (usually looks like https://bucket-name.s3.amazonaws.com/filename).
4. Copy the URL and paste it into a **new Incognito/Private browser window**.
5. If the file opens or downloads, you have successfully completed the task!



Q4 Using Amazon Web Services, create and configure a relational database instance using Amazon RDS.

Step 1: Open Amazon RDS

1. Log in to the **AWS Management Console**.
2. Search for **RDS** and select it from the services list.



Step 2 & 3: Create Database (Standard Create & Engine)

1. Click **Create database**.
2. Choose **Full configuration create** (do not use "Easy create").
3. **Engine options:** Select **MySQL** (this is usually easiest for practicals).
4. **Templates:** Select **Free Tier**. This is crucial—it will automatically disable expensive options.

Step 4 & 5: Configure Instance & Credentials

Once you click "Full configuration", follow these exact steps:

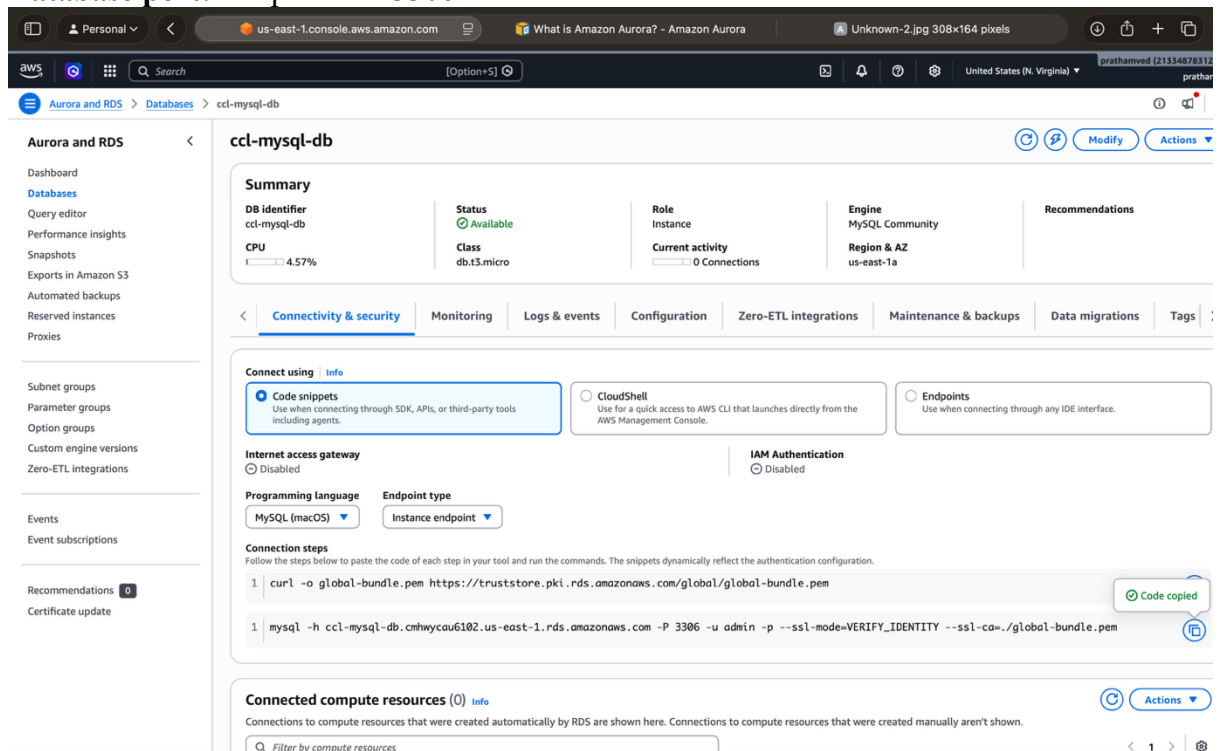
1. **Engine Options:** Choose **MySQL**.
2. **Templates:** Select **Free Tier** (this simplifies the rest of the options for you).
3. **Settings (Task 5):**
 - Set **Master username** (e.g., admin).
 - Set **Master password** (e.g., CclLab123).
4. **Instance configuration (Task 4):** Ensure it is set to **db.t3.micro** (Standard Free Tier)

Step 6: Storage (Free Tier Limits)

1. **Allocated storage:** Set this to **20 GiB** (the Free Tier limit).
2. **Storage autoscaling:** **Uncheck** "Enable storage autoscaling" to stay strictly within limits.

Step 7: Connectivity (Public Access)

1. **Public access:** Select **Yes**. (This is required for Task 7 and 9).
2. **VPC security group:** Choose **Create new**.
3. **New VPC security group name:** ccl-rds-sg.
4. **Database port:** Keep default 3306.



Step 8: Automatic Backups

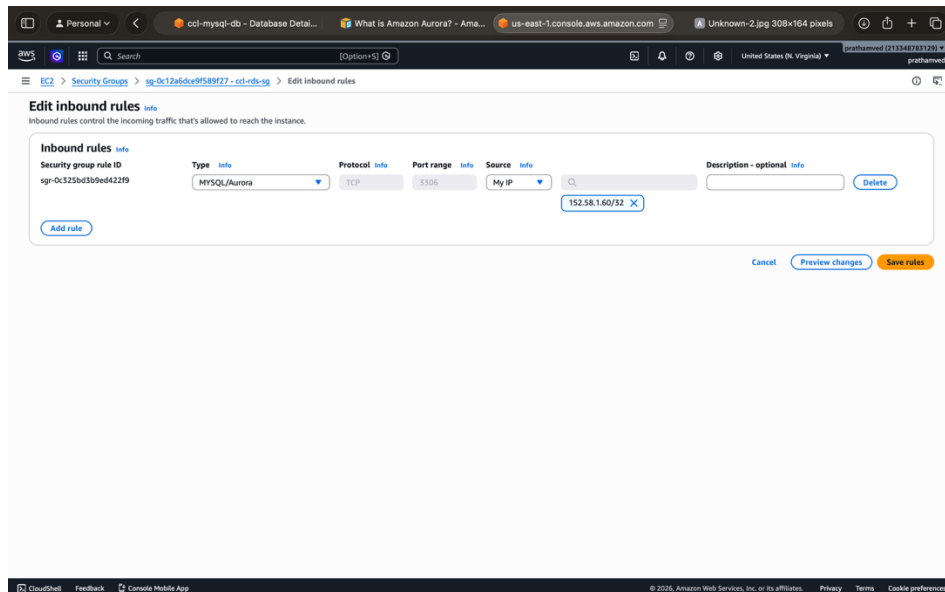
1. Expand the **Additional configuration** section at the bottom.
2. Under **Backup**, ensure **Enable automatic backups** is checked.
3. Set the **Backup retention period** to **1 day** (the minimum for the lab).
4. Click **Create database**.
 - *Note: It takes about 5–10 minutes for the status to change from "Creating" to "Available."*

Step 9: Connect to the Database

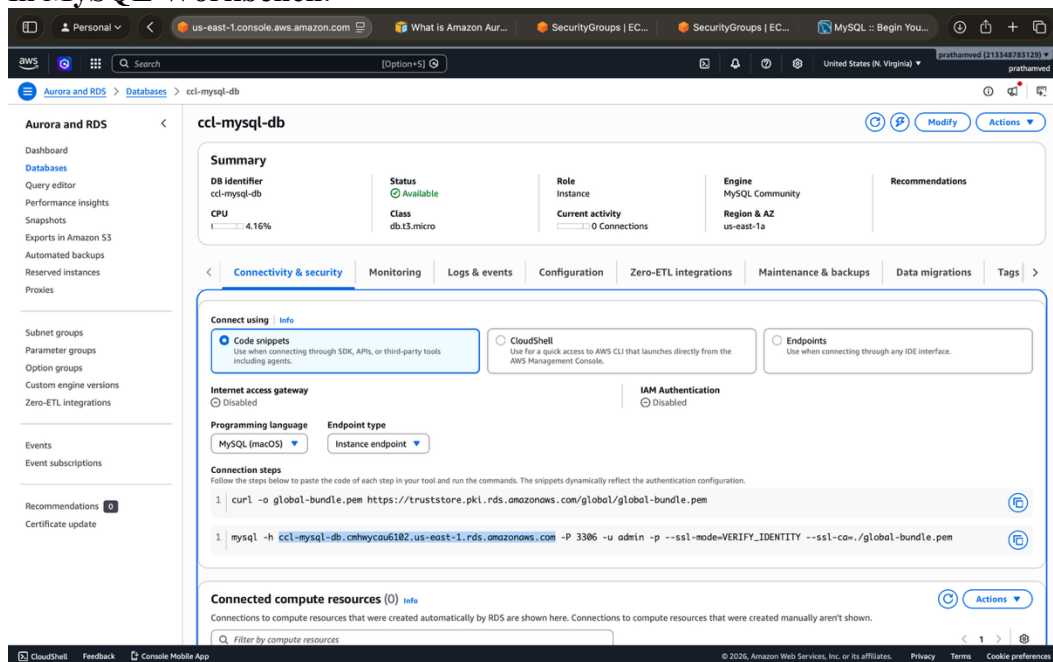
To connect from your computer (MySQL Workbench):

1. Click on your database name in the RDS console.
2. Copy the **Endpoint** (it looks like ccl-mysql-db.xxxx.us-east-1.rds.amazonaws.com).
3. **Important Security Step:**

- Click on your **VPC security group** (under the Connectivity tab).
- Edit **Inbound rules**.
- Add a rule: **Type:** MySQL/Aurora | **Port:** 3306 | **Source:** My IP.
- Click **Save rules**.



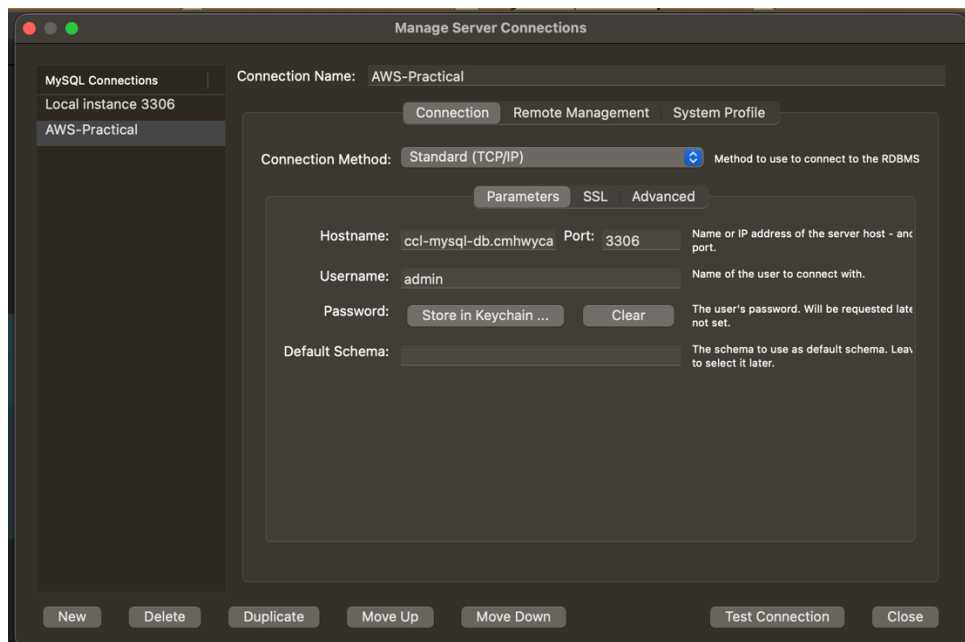
4. In MySQL Workbench:



1. Fill in the Details

- **Connection Name:** Type AWS-Practical.
- **Hostname:** Delete 127.0.0.1 and paste your **RDS Endpoint**.
 - It should look something like: `ccl-mysql-db.xxxx.us-east-1.rds.amazonaws.com`.
- **Username:** Change root to admin.
- **Password:** Click **Store in Keychain...** and type the password you set during the RDS creation (the one you used in Step 5).

- **Default Schema:** Leave this blank for now.



Step 10: Verify Connectivity

Once connected, run these SQL commands:

SQL

-- Create a sample table

```
CREATE DATABASE ccl_test;
```

```
USE ccl_test;
```

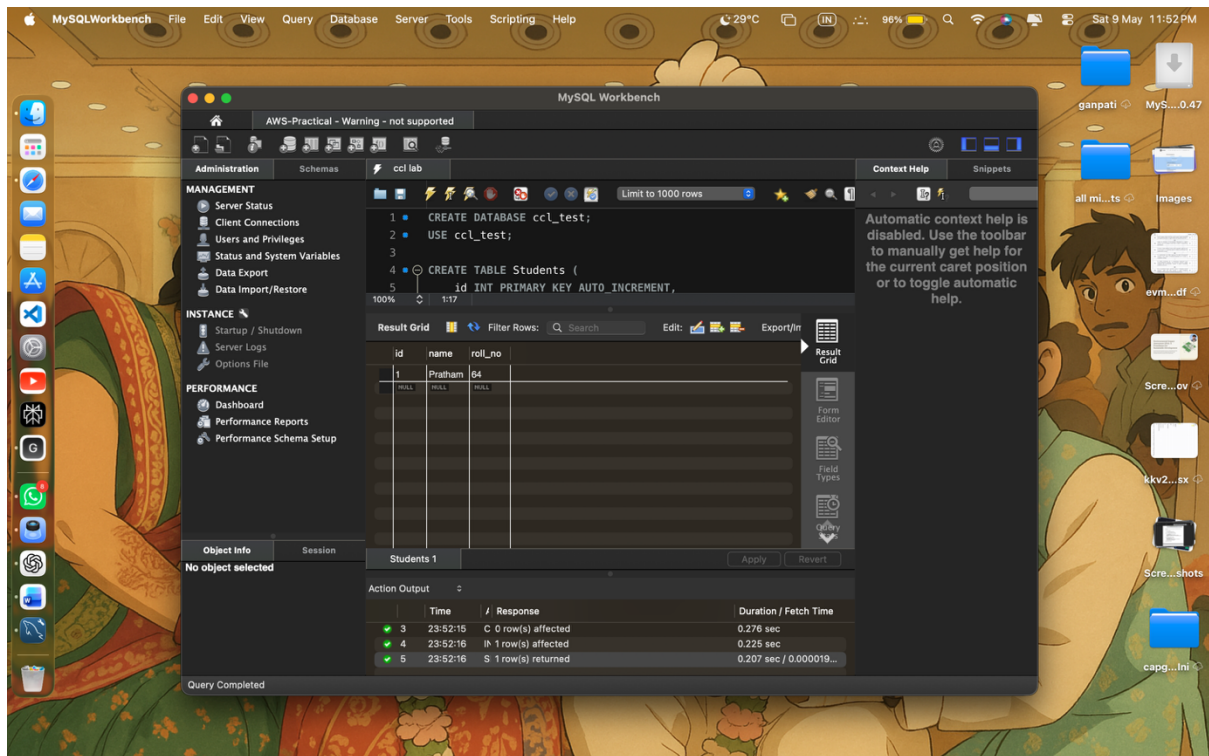
```
CREATE TABLE Students (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  name VARCHAR(50),  
  roll_no INT  
);
```

-- Insert a record

```
INSERT INTO Students (name, roll_no) VALUES ('Pratham', 64);
```

-- Verify

```
SELECT * FROM Students;
```



Q5 Using Amazon Web Services, create and deploy a simple static website using Amazon S3.

Tasks to be Performed

Step 1 & 2: Create Bucket and Configure Hosting

1. **Open S3** and click **Create bucket**.
2. **Bucket name:** Use something unique like my-static-site-pratham-64.
3. **Region:** Keep it as us-east-1.
4. **Public Access:** **Uncheck** "Block all public access" and acknowledge the warning.
5. **Create bucket**.
6. Click on your new bucket -> **Properties** tab.
7. Scroll to the very bottom to **Static website hosting** and click **Edit**.
8. Select **Enable**.
9. **Index document:** Type index.html.
10. Click **Save changes**.

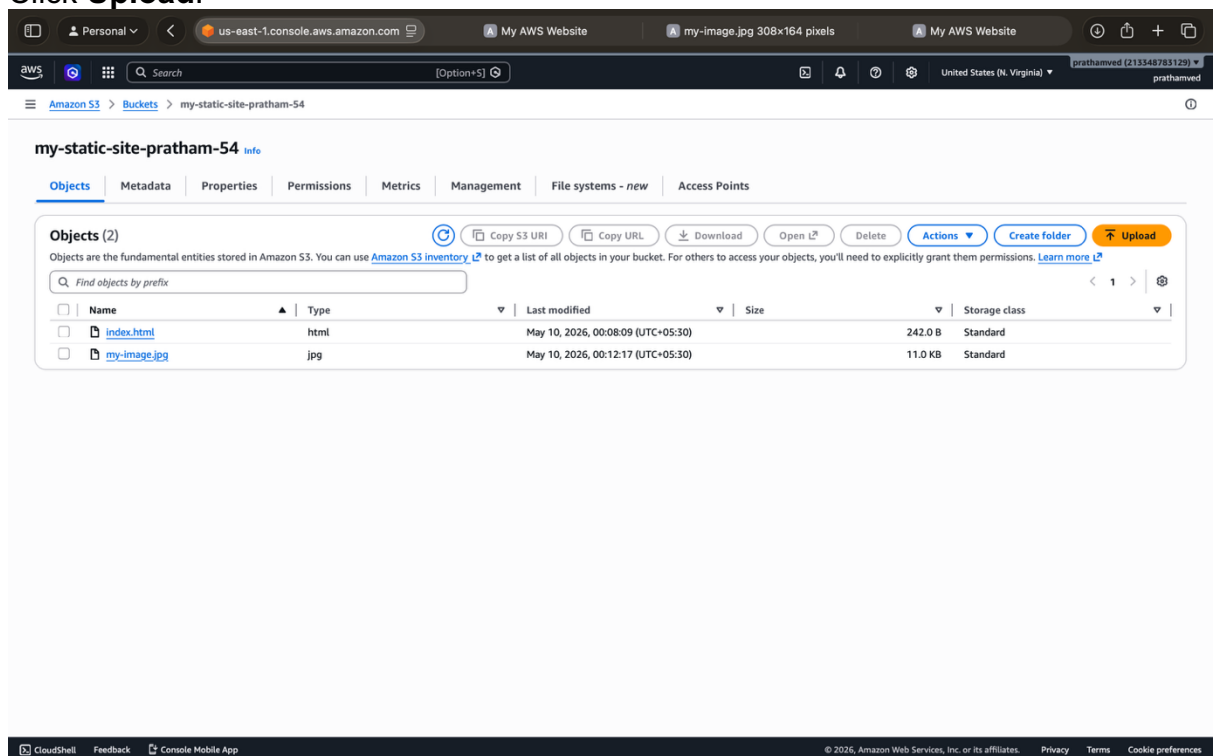
Step 3: Prepare and Upload Files

On your computer, create a simple file named index.html using Notepad or VS Code:

HTML

```
<!DOCTYPE html>
<html>
<head>
  <title>My AWS Website</title>
</head>
<body>
  <h1>Welcome to my S3 Static Website</h1>
  <p>This site is hosted on Amazon S3!</p>
  
</body>
</html>
```

1. In the **Objects** tab of your bucket, click **Upload**.
2. Add your index.html and an image file (make sure the image name matches exactly what you wrote in the code, e.g., my-image.jpg).
3. Click **Upload**.



Step 4: Enable Public Access (Bucket Policy)

1. Go to the **Permissions** tab.
2. Scroll to **Bucket policy** and click **Edit**.
3. Paste this JSON (replace YOUR-BUCKET-NAME with your actual bucket name):

JSON

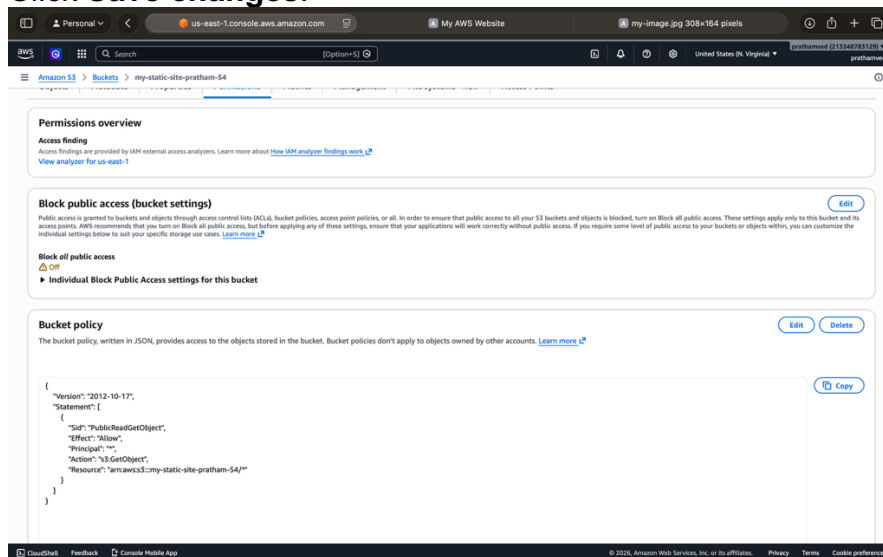
```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "PublicReadGetObject",
      "Effect": "Allow",
      "Principal": "*",
      "Action": "s3:GetObject",
```

```

    "Resource": "arn:aws:s3:::YOUR-BUCKET-NAME/*"
  }
}
}

```

4. Click **Save changes**.

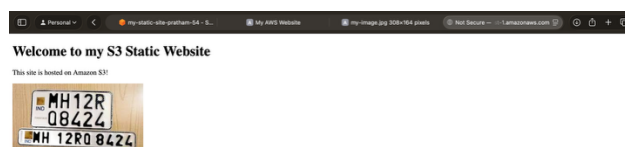


Step 5, 6 & 7: Access and Verify

1. Go back to the **Properties** tab.
2. Scroll to the bottom under **Static website hosting**.
3. You will see a link called **Bucket website endpoint**. It looks like:

http://your-bucket-name.s3-website-us-east-1.amazonaws.com

4. **Click the link.**
5. Your website should load showing your heading, text, and the image.



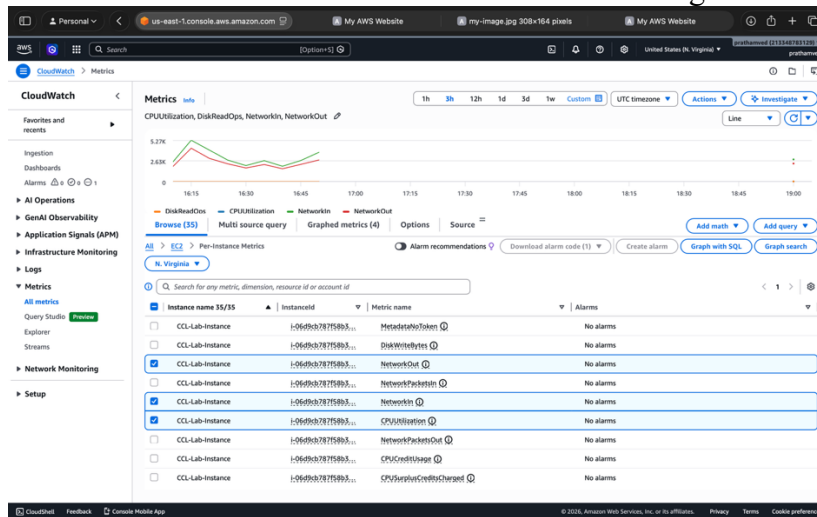
Step 1: Launch and Access CloudWatch

1. **Launch Instance:** Ensure you have a running EC2 instance (Amazon Linux).
2. **Open CloudWatch:** Search for **CloudWatch** in the AWS Console and open it.
3. **View Metrics:** In the left menu, click **Metrics** -> **All metrics**.
4. **Find EC2 Data:** Click on **EC2** -> **Per-Instance Metrics**. Search for your Instance ID to see the specific graphs for your server.

Step 2: Monitor Key Metrics

Under the **Graphed metrics** tab, you can visualize the following as required by your task:

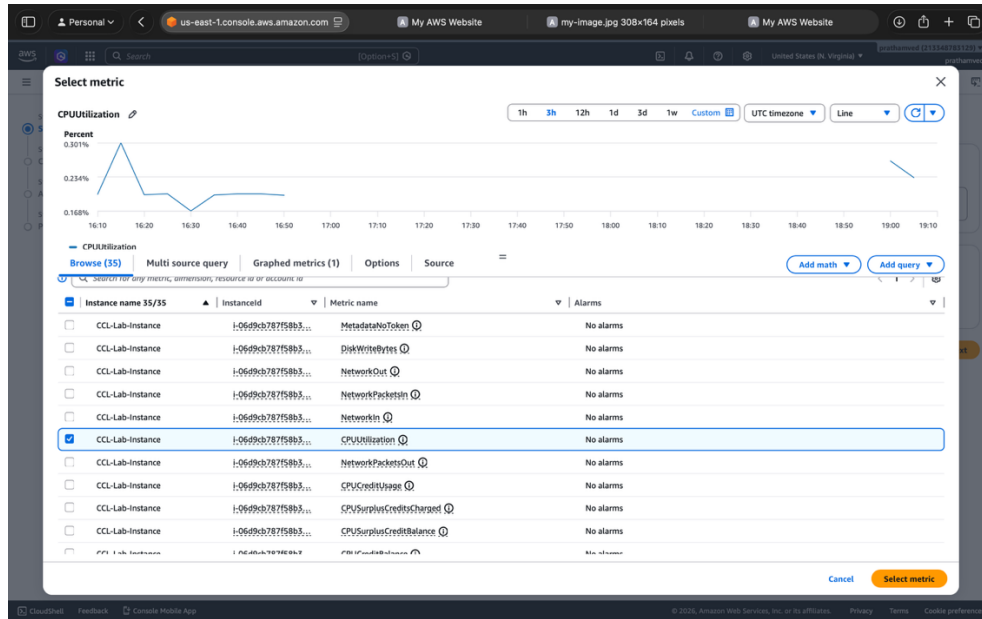
- **CPUUtilization:** Percentage of allocated compute power being used.
- **DiskReadOps:** Number of read operations from the EBS volume.
- **NetworkIn / NetworkOut:** Volume of traffic flowing into and out of the instance.



Step 3: Create a CloudWatch Alarm (70% CPU Trigger)

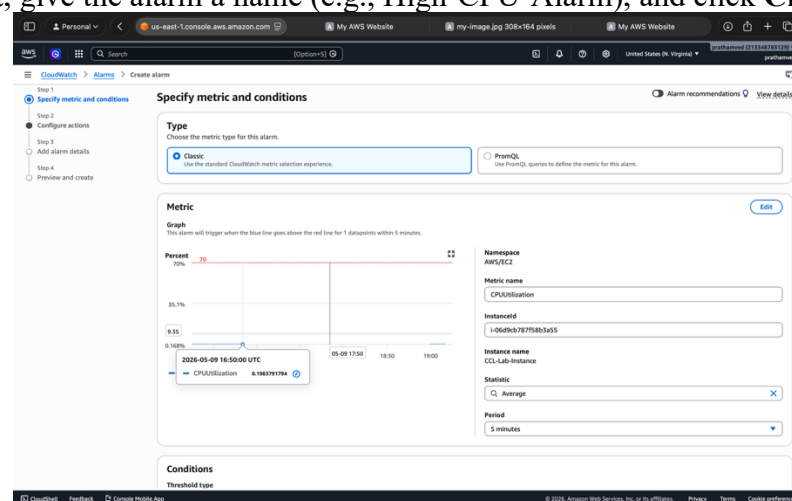
1. In the CloudWatch left menu, click **Alarms** -> **All alarms** -> **Create alarm**.
2. Click **Select metric**.
3. Navigate to **EC2** -> **Per-Instance Metrics** and check the box for your instance where the Metric Name is **CPUUtilization**. Click **Select metric**.
4. **Conditions:**
 - **Threshold type:** Static.
 - **Whenever CPUUtilization is...:** Greater.
 - **than...:** 70.

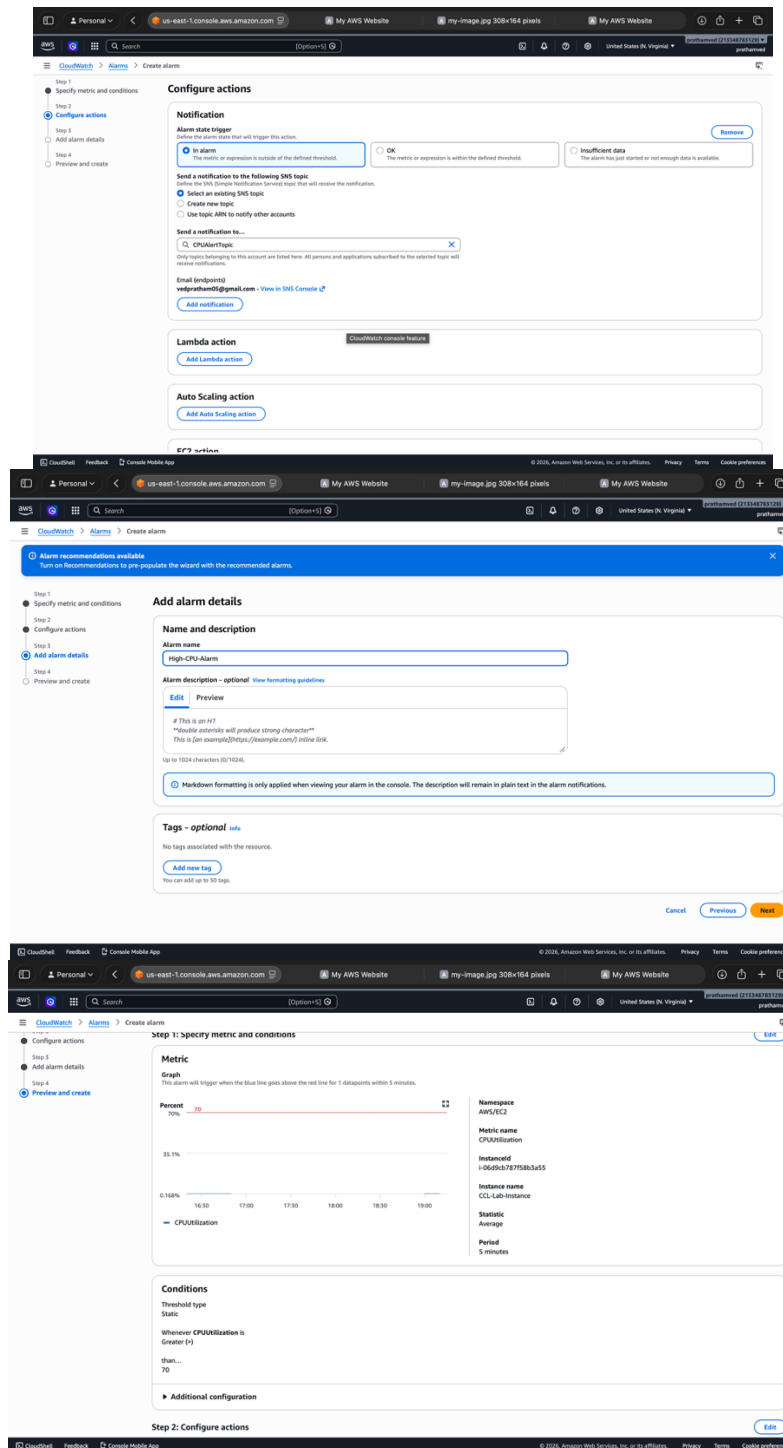
5. Click Next.



Step 4: Configure Notifications (SNS)

1. **Alarm state trigger:** Select **In alarm**.
2. **Send a notification to the following SNS topic:** Select **Create new topic**.
3. **Topic Name:** CPU_Alert_Topic.
4. **Email endpoints:** Enter your email address.
5. Click **Create topic**.
 - *Note: Check your email inbox and click "Confirm Subscription" in the email AWS sends you, otherwise the alert won't reach you.*
6. Click **Next**, give the alarm a name (e.g., High-CPU-Alarm), and click **Create alarm**.





Step 5: Verify Alarm (Generate Load)

Connect to your Instance

1. Go back to your **EC2 Instances** list.
2. Select your running instance (i-06d9cb787f58b3a55).
3. Click the **Connect** button at the top of the screen.
4. Choose the **EC2 Instance Connect** tab.

- Click the orange **Connect** button at the bottom right. A new browser tab will open with a black terminal screen.

To verify the alarm, you need to connect to your instance using the **terminal** so you can run that stress command.

Step 1: Connect to your Instance

- Go back to your **EC2 Instances** list.
- Select your running instance (i-06d9cb787f58b3a55).
- Click the **Connect** button at the top of the screen.
- Choose the **EC2 Instance Connect** tab.
- Click the orange **Connect** button at the bottom right. A new browser tab will open with a black terminal screen.

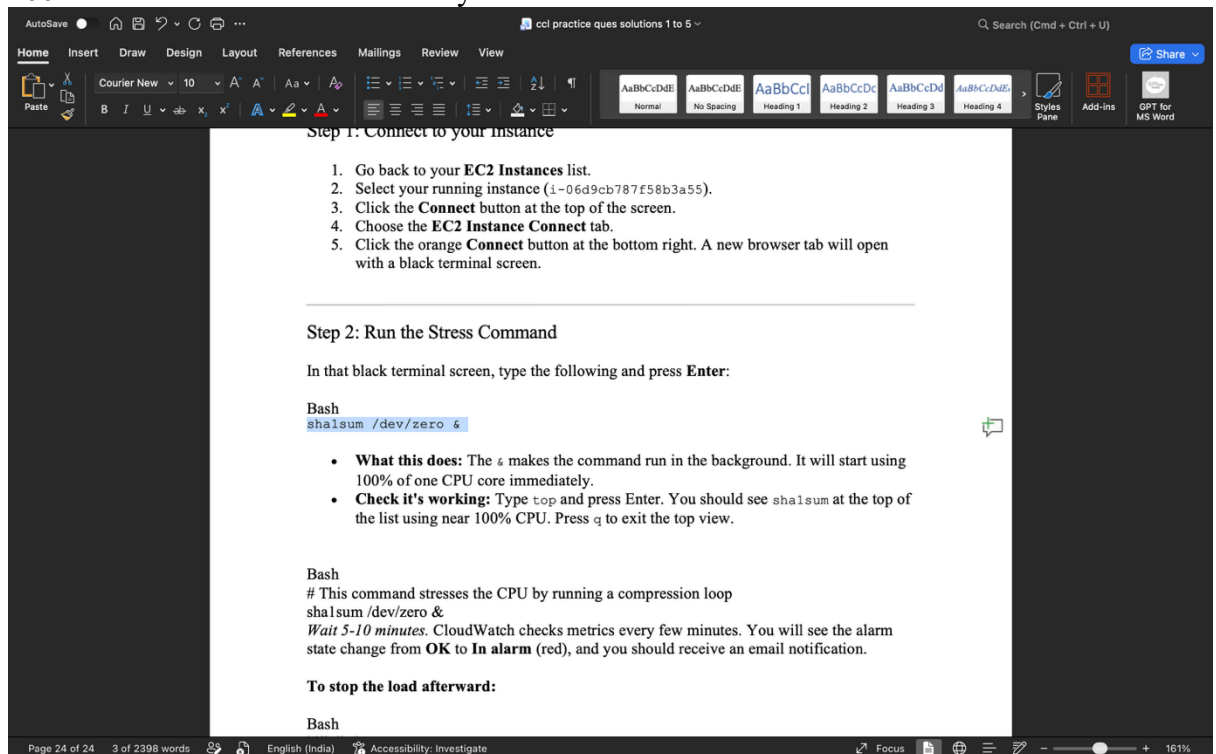
Step 2: Run the Stress Command

In that black terminal screen, type the following and press **Enter**:

Bash

```
shalsum /dev/zero &
```

- What this does:** The `&` makes the command run in the background. It will start using 100% of one CPU core immediately.



- Check it's working:** Type `top` and press Enter. You should see `shalsum` at the top of the list using near 100% CPU. Press `q` to exit the top view.

```
top - 19:26:23 up 27 min, 1 user, load average: 1.19, 0.37, 0.13
Tasks: 101 total, 3 running, 98 sleeping, 0 stopped, 0 zombie
rtpm(s): 95.3 us, 4.7 sy, 0.0 ni, 0.0 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 961.3 total, 634.1 free, 114.3 used, 212.9 buff/cache
MiB Swap: 0.0 total, 0.0 free, 0.0 used, 709.6 avail Mem

  PID USER      PR  NI    VIRT    RES    SHR   S  %CPU  %MEM     TIME+ COMMAND
 3228 ec2-user    20   0 227464   2596   2012  R   50.0   0.3   1:15.29 shalsu
3231 ec2-user    20   0 227464   2592   2008  R   49.7   0.3   0:15.27 shalsu
   1 root        20   0 107496  17440  10900  S   0.0   1.8   0:00.99 systemd
   2 root        20   0      0      0      0  S   0.0   0.0   0:00.00 kthreadd
   3 root        0 -20   0      0      0  I   0.0   0.0   0:00.00 rcu_gp
   4 root        0 -20   0      0      0  I   0.0   0.0   0:00.00 rcu_par_gp
   5 root        0 -20   0      0      0  I   0.0   0.0   0:00.00 slub_flushwq
   6 root        0 -20   0      0      0  I   0.0   0.0   0:00.00 netns
   8 root        0 -20   0      0      0  I   0.0   0.0   0:00.00 kworker/0:0H-events_highpri
  10 root        0 -20   0      0      0  I   0.0   0.0   0:00.00 mm_percpu_wq
  11 root        20   0      0      0      0  I   0.0   0.0   0:00.00 rcu_tasks_kthread
  12 root        20   0      0      0      0  I   0.0   0.0   0:00.00 rcu_tasks_rude_kthread
  13 root        20   0      0      0      0  I   0.0   0.0   0:00.00 rcu_tasks_trace_kthread
  14 root        20   0      0      0      0  S   0.0   0.0   0:00.03 ksoftirqd/0
  15 root        20   0      0      0      0  I   0.0   0.0   0:00.04 rcu_preempt
  16 root        20   0      0      0      0  S   0.0   0.0   0:00.00 migration/0
  18 root        20   0      0      0      0  S   0.0   0.0   0:00.00 gpaph/0
  20 root        20   0      0      0      0  S   0.0   0.0   0:00.00 kdevtmpfs
  21 root        0 -20   0      0      0  I   0.0   0.0   0:00.00 inet_frag_wq
  22 root        20   0      0      0      0  S   0.0   0.0   0:00.00 kauditd
  23 root        20   0      0      0      0  S   0.0   0.0   0:00.00 khungtaskd
  24 root        20   0      0      0      0  S   0.0   0.0   0:00.00 oom_reaper
  26 root        20   0      0      0      0  I   0.0   0.0   0:00.03 kworker/u30:2-events_unbound
  27 root        0 -20   0      0      0  I   0.0   0.0   0:00.00 writeback
  28 root        20   0      0      0      0  S   0.0   0.0   0:00.02 kcompactd0
  29 root        39  19   0      0      0  S   0.0   0.0   0:00.00 khugepaged
  30 root        0 -20   0      0      0  I   0.0   0.0   0:00.00 cryptd
  31 root        0 -20   0      0      0  I   0.0   0.0   0:00.00 kintegrityd
  32 root        0 -20   0      0      0  I   0.0   0.0   0:00.00 kblockd
  33 root        0 -20   0      0      0  I   0.0   0.0   0:00.00 blkcg_punt_bio
  34 root        20   0      0      0      0  S   0.0   0.0   0:00.00 xen-balloon
  35 root        0 -20   0      0      0  I   0.0   0.0   0:00.00 cpu_dev_wq
  36 root        0 -20   0      0      0  I   0.0   0.0   0:00.00 md
  37 root        0 -20   0      0      0  I   0.0   0.0   0:00.00 edac-poller
  38 root       -51   0      0      0      0  S   0.0   0.0   0:00.00 watchdogd
  39 root        0 -20   0      0      0  I   0.0   0.0   0:00.01 kworker/0:1H-kblockd
  74 root        20   0      0      0      0  S   0.0   0.0   0:00.00 kswapd0
  77 root        0 -20   0      0      0  I   0.0   0.0   0:00.00 xfsalloc
```

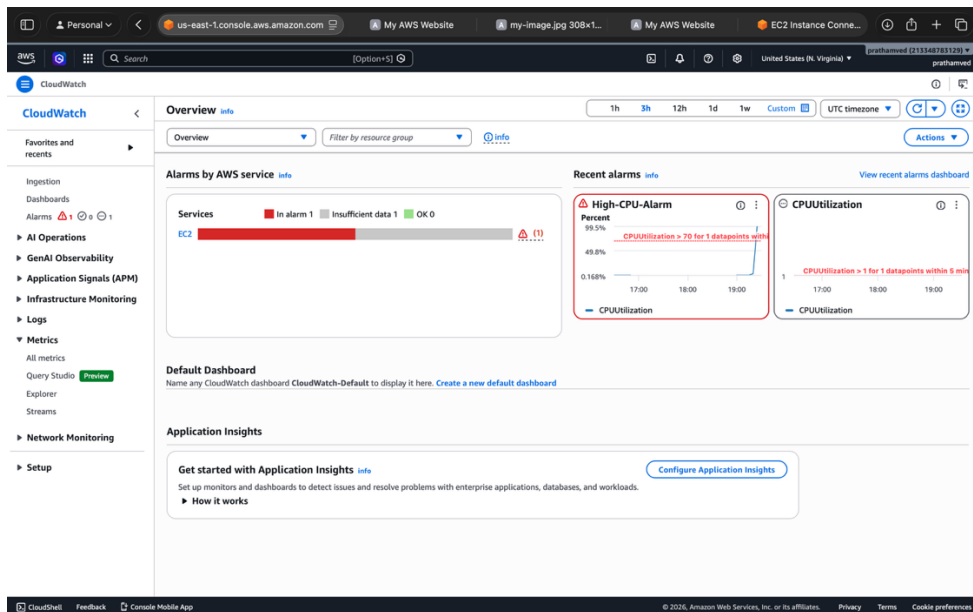
i-06d9cb78758b3a55 (CCL-Lab-Instance)
PublicIPs: 100.48.208.231 PrivateIPs: 172.31.0.147

Bash

This command stresses the CPU by running a compression loop

shalsu /dev/zero &

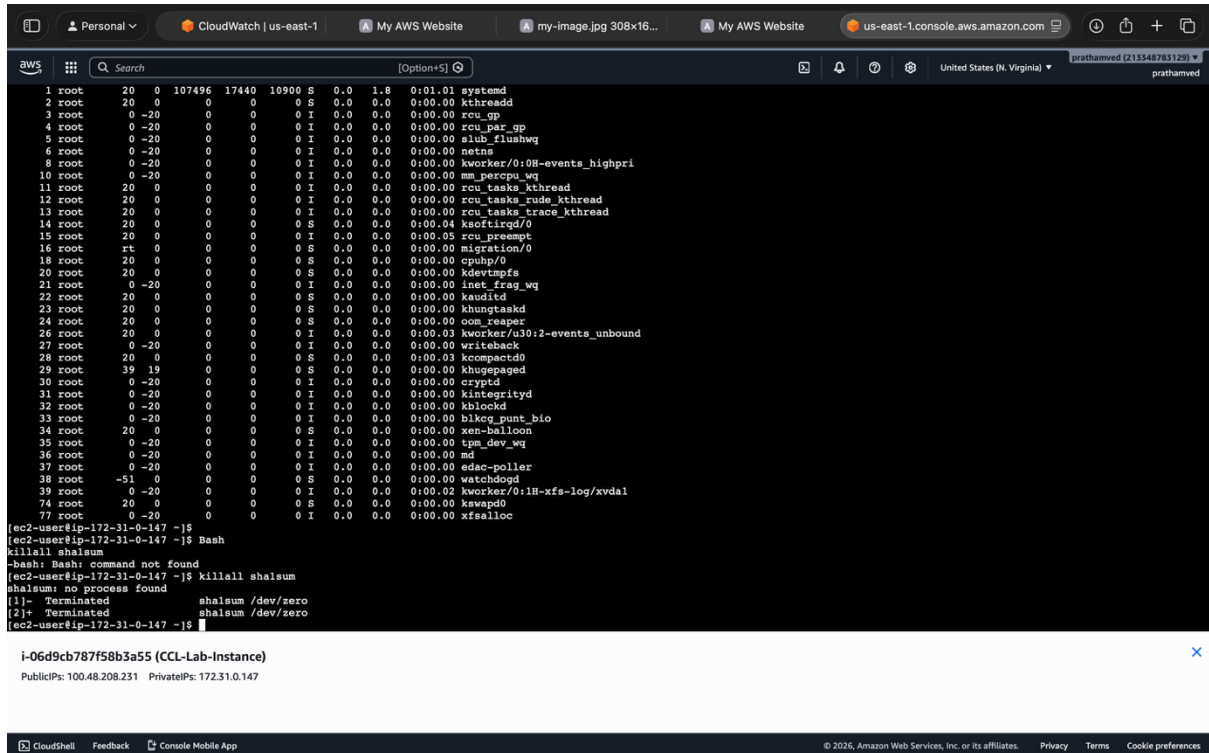
Wait 5-10 minutes. CloudWatch checks metrics every few minutes. You will see the alarm state change from **OK** to **In alarm** (red), and you should receive an email notification.



To stop the load afterward:

Bash

killall shalsum



```
1 root      20    0 107496 17440 10900 S   0.0  1.8  0:01.01 systemd
2 root      20    0      0      0      0 S   0.0  0.0  0:00.00 kthreadd
3 root      0 -20    0      0      0 I   0.0  0.0  0:00.00 rcu_gp
4 root      0 -20    0      0      0 I   0.0  0.0  0:00.00 rcu_par_gp
5 root      0 -20    0      0      0 I   0.0  0.0  0:00.00 slub_flushwq
6 root      0 -20    0      0      0 I   0.0  0.0  0:00.00 netns
8 root      0 -20    0      0      0 I   0.0  0.0  0:00.00 kworker/0:0m-events_highpri
10 root     0 -20    0      0      0 I   0.0  0.0  0:00.00 mm_percpu_wq
11 root     20    0      0      0      0 I   0.0  0.0  0:00.00 rcu_tasks_kthread
12 root     20    0      0      0      0 I   0.0  0.0  0:00.00 rcu_tasks_rude_kthread
13 root     20    0      0      0      0 I   0.0  0.0  0:00.00 rcu_tasks_trace_kthread
14 root     20    0      0      0      0 S   0.0  0.0  0:00.00 ksortirq/0
15 root     20    0      0      0      0 I   0.0  0.0  0:00.05 rcu_preempt
16 root     rt    0      0      0      0 S   0.0  0.0  0:00.00 migration/0
18 root     20    0      0      0      0 S   0.0  0.0  0:00.00 cpuhp/0
20 root     20    0      0      0      0 S   0.0  0.0  0:00.00 kdevtmpfs
21 root     0 -20    0      0      0 I   0.0  0.0  0:00.00 inet_frag_wq
22 root     20    0      0      0      0 S   0.0  0.0  0:00.00 kauditd
23 root     20    0      0      0      0 S   0.0  0.0  0:00.00 khungtaskd
24 root     20    0      0      0      0 S   0.0  0.0  0:00.00 oom_reaper
26 root     20    0      0      0      0 I   0.0  0.0  0:00.03 kworker/u3:0-2-events_unbound
27 root     0 -20    0      0      0 I   0.0  0.0  0:00.00 writeback
28 root     20    0      0      0      0 S   0.0  0.0  0:00.03 kcompactd0
29 root     39 19    0      0      0 S   0.0  0.0  0:00.00 khugepaged
30 root     0 -20    0      0      0 I   0.0  0.0  0:00.00 cryptd
31 root     0 -20    0      0      0 I   0.0  0.0  0:00.00 kintegrityd
32 root     0 -20    0      0      0 I   0.0  0.0  0:00.00 kblockd
33 root     0 -20    0      0      0 I   0.0  0.0  0:00.00 hikjey_punt_bio
34 root     20    0      0      0      0 S   0.0  0.0  0:00.00 xen-balloon
35 root     0 -20    0      0      0 I   0.0  0.0  0:00.00 tpm_dev_wq
36 root     0 -20    0      0      0 I   0.0  0.0  0:00.00 md
37 root     0 -20    0      0      0 I   0.0  0.0  0:00.00 mdadm-poller
38 root     -51    0      0      0      0 S   0.0  0.0  0:00.00 watchdogd
39 root     0 -20    0      0      0 I   0.0  0.0  0:00.02 kworker/0:1H-xfs-log/xvdal
74 root     20    0      0      0      0 S   0.0  0.0  0:00.00 kswapd0
77 root     0 -20    0      0      0 I   0.0  0.0  0:00.00 xfsalloc

[ec2-user@ip-172-31-0-147 ~]$
[ec2-user@ip-172-31-0-147 ~]$ Bash
killall shalsum
-bash: killall: command not found
[ec2-user@ip-172-31-0-147 ~]$ killall shalsum
shalsum: no process found
[1]- Terminated                  shalsum /dev/zero
[2]+ Terminated                  shalsum /dev/zero
[ec2-user@ip-172-31-0-147 ~]$
```

i-06d9cb787f58b3a55 (CCL-Lab-Instance)
PublicIPs: 100.48.208.231 PrivateIPs: 172.31.0.147

Q7

Since you have your **CCL-Lab-Instance** already running, you can actually use it to complete this final task. You've already done the hard part of connecting to the terminal; now you just need to turn it into a web server and handle the resizing.

Step 1: Use Your Existing Instance

You are already using an **Amazon Linux** instance (i-06d9cb787f58b3a55). Make sure it is in the **Running** state.

Step 2 & 3: Install and Start Apache

Connect to your instance via **EC2 Instance Connect** (the terminal you used for the stress test) and run these commands one by one:

Bash

```
# Update the package manager
sudo yum update -y
```

```
# Install Apache (httpd)
sudo yum install -y httpd
```

```
[ec2-user@ip-172-31-0-147 ~]$ sudo yum update -y
Amazon Linux 2023 Kernel Livepatch repository
Dependencies resolved.
Nothing to do.
Complete!
[ec2-user@ip-172-31-0-147 ~]$ sudo yum install -y httpd
Last metadata expiration check: 0:00:50 ago on Sat May 9 19:35:41 2026.
Dependencies resolved.

=====
Package                                Architecture      Version            Repository          Size
=====
Installing:
httpd                                  x86_64            2.4.66-1.amzn2023.0.1  amazonlinux        47 k
Installing dependencies:
apr                                    x86_64            1.7.5-1.amzn2023.0.4  amazonlinux        129 k
apr-util                              x86_64            1.6.3-1.amzn2023.0.2  amazonlinux        97 k
apr-util-ldap                         x86_64            1.6.3-1.amzn2023.0.2  amazonlinux        13 k
generic-logos-httpd                  noarch            18.0.0-12.amzn2023.0.3  amazonlinux        19 k
httpd-core                           x86_64            2.4.66-1.amzn2023.0.1  amazonlinux        1.4 M
httpd-filesystem                     noarch            2.4.66-1.amzn2023.0.1  amazonlinux        13 k
httpd-tools                          x86_64            2.4.66-1.amzn2023.0.1  amazonlinux        81 k
libbrotli                             x86_64            1.0.9-4.amzn2023.0.2  amazonlinux        315 k
mailcap                              noarch            2.1.49-3.amzn2023.0.3  amazonlinux        33 k
Installing weak dependencies:
apr-util-openssl                     x86_64            1.6.3-1.amzn2023.0.2  amazonlinux        15 k
mod_http2                            x86_64            2.0.27-1.amzn2023.0.3  amazonlinux        166 k
mod_lua                              x86_64            2.4.66-1.amzn2023.0.1  amazonlinux        60 k
=====
Transaction Summary
=====
Install 13 Packages

Total download size: 2.4 M
Installed size: 6.9 M
Downloading Packages:
(1/13): apr-util-ldap-1.6.3-1.amzn2023.0.2.x86_64.rpm                278 kB/s | 13 kB  00:00
(2/13): apr-1.7.5-1.amzn2023.0.4.x86_64.rpm                        2.4 MB/s | 129 kB  00:00
(3/13): apr-util-1.6.3-1.amzn2023.0.2.x86_64.rpm                  1.7 MB/s | 97 kB  00:00
(4/13): apr-util-openssl-1.6.3-1.amzn2023.0.2.x86_64.rpm          531 kB/s | 15 kB  00:00
(5/13): generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch.rpm      674 kB/s | 19 kB  00:00
(6/13): httpd-2.4.66-1.amzn2023.0.1.x86_64.rpm                   1.6 MB/s | 47 kB  00:00
(7/13): httpd-filesystem-2.4.66-1.amzn2023.0.1.noarch.rpm          477 kB/s | 13 kB  00:00
(8/13): httpd-tools-2.4.66-1.amzn2023.0.1.x86_64.rpm              2.8 MB/s | 81 kB  00:00
(9/13): httpd-core-2.4.66-1.amzn2023.0.1.x86_64.rpm               31 MB/s | 1.4 MB  00:00
(10/13): libbrotli-1.0.9-4.amzn2023.0.2.x86_64.rpm                11 MB/s | 315 kB  00:00
(11/13): mailcap-2.1.49-3.amzn2023.0.3.noarch.rpm                 2.1 MB/s | 33 kB  00:00
(12/13): apr-util-openssl-1.6.3-1.amzn2023.0.2.x86_64.rpm         1.6 MB/s | 15 kB  00:00
(13/13): mod_http2-2.0.27-1.amzn2023.0.3.x86_64.rpm              2.0 MB/s | 166 kB  00:00

i-06d9cb787f58b3a55 (CCL-Lab-Instance)
PublicIPs: 100.48.208.231 PrivateIPs: 172.31.0.147
```

Start the service

sudo systemctl start httpd

Enable it to start automatically on reboot

sudo systemctl enable httpd

Create a simple landing page

echo "<h1>Welcome to Pratham's EC2 Web Server</h1>" | sudo tee /var/www/html/index.html

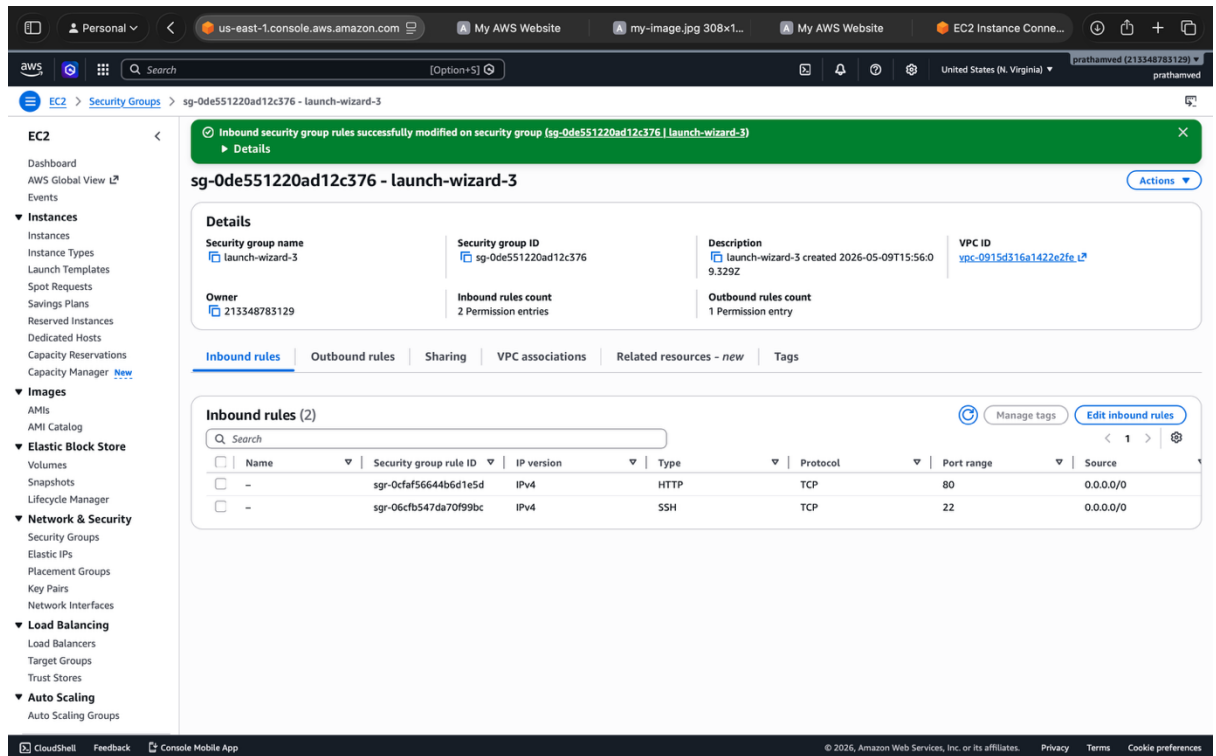
```
[ec2-user@ip-172-31-0-147 ~]$ sudo systemctl start httpd
[ec2-user@ip-172-31-0-147 ~]$ sudo systemctl enable httpd
Created symlink /etc/systemd/system/multi-user.target.wants/httpd.service - /usr/lib/systemd/system/httpd.service.
[ec2-user@ip-172-31-0-147 ~]$ echo "<h1>Welcome to Pratham's EC2 Web Server</h1>" | sudo tee /var/www/html/index.html
[ec2-user@ip-172-31-0-147 ~]$
```

Step 4: Allow HTTP Access

You already know how to edit **Security Groups**!

1. Go to your instance's **Security Groups**.
2. Edit **Inbound Rules**.

3. Add a new rule: **Type:** HTTP | **Port:** 80 | **Source:** Anywhere-IPv4 (0.0.0.0/0).
4. **Save rules.**



sg-0de551220ad12c376 - launch-wizard-3

Details

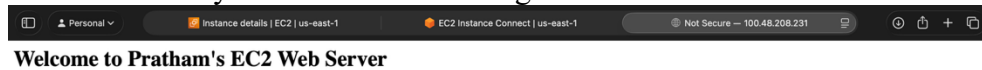
Security group name launch-wizard-3	Security group ID sg-0de551220ad12c376	Description launch-wizard-3 created 2026-05-09T15:56:09.329Z	VPC ID vpc-0915d316a1422e2fe
Owner 213348783129	Inbound rules count 2 Permission entries	Outbound rules count 1 Permission entry	

Inbound rules (2)

Name	Security group rule ID	IP version	Type	Protocol	Port range	Source
-	sgr-0cfaf56644b6d1e5d	IPv4	HTTP	TCP	80	0.0.0.0/0
-	sgr-06cfb547da70f99bc	IPv4	SSH	TCP	22	0.0.0.0/0

Step 5: Verify the Server

1. Copy the **Public IPv4 address** of your instance (e.g., 100.48.208.231).
2. Paste it into your browser's address bar.
3. You should see your "Welcome" message.



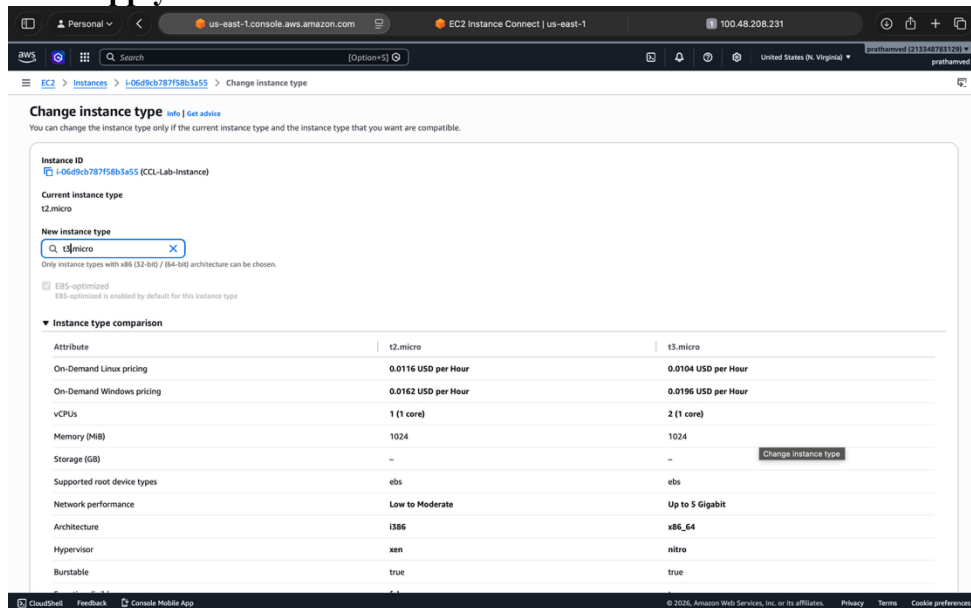
Welcome to Pratham's EC2 Web Server

Step 6: Resize the Instance (The "Resource Modification" Part)

This is a two-part task that requires the instance to be **Stopped** first.

Part A: Changing Instance Type

1. Go to the EC2 Dashboard and **Stop** your instance.
2. Once stopped, click **Actions** -> **Instance Settings** -> **Change instance type**.
3. Change it from t2.micro to t3.micro (or any other type allowed in your lab).
4. Click **Apply**.



Part B: Modifying EBS Storage

1. In the left menu, go to **Elastic Block Store** -> **Volumes**.
2. Select the volume attached to your instance.
3. Click **Actions** -> **Modify volume**.
4. Change the **Size** (e.g., if it's 8 GiB, change it to 10 GiB).
5. Click **Modify** and confirm.
6. Go back to **Instances** and **Start** your instance again.

Q8

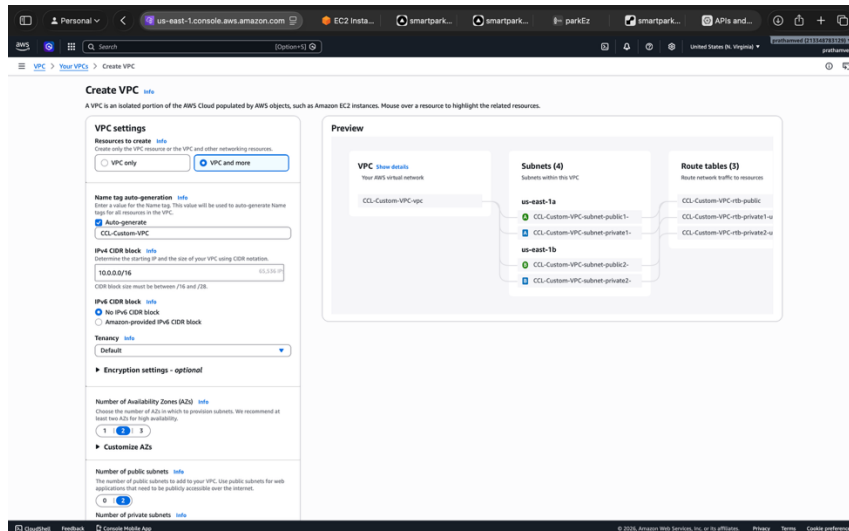
This is a more advanced networking task that builds on everything you've done tonight. It essentially asks you to build a small, secure data center in the cloud.

Step 1: Create the VPC with Subnets (The Network Foundation)

Instead of creating everything manually, the **VPC Wizard** is the fastest way to get this right for a lab.

1. Search for **VPC** in the console.
2. Click **Create VPC**.

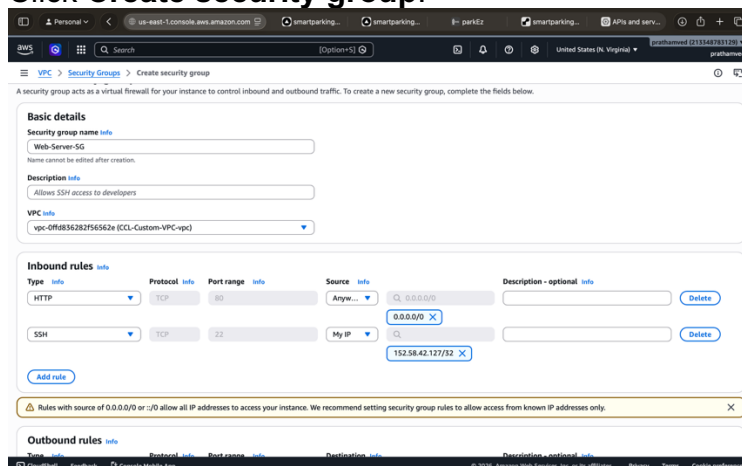
3. Select **VPC and more** (this creates your subnets, gateways, and routing in one go).
4. **Name tag:** CCL-Custom-VPC.
5. **Availability Zones (AZs):** Select 2.
6. **Number of Public Subnets:** Select 2.
7. **Number of Private Subnets:** Select 2.
8. **NAT Gateway:** Select **In 1 AZ** (this allows private subnets to talk to the internet for updates, but stays hidden).
9. Click **Create VPC**.



Step 2: Security Settings (The Firewalls)

You need a Security Group that specifically allows web traffic.

1. In the VPC left menu, click **Security Groups -> Create security group**.
2. **Name:** Web-Server-SG.
3. **VPC:** Ensure you select your CCL-Custom-VPC.
4. **Inbound Rules:**
 - **Type:** HTTP | **Port:** 80 | **Source:** 0.0.0.0/0
 - **Type:** SSH | **Port:** 22 | **Source:** My IP (for your own access).
5. Click **Create security group**.



Step 3: Launch Web Server with User Data (Automation)

Instead of installing Apache manually like last time, we will use a **User Data script** to do it automatically during launch.

1. Go to **EC2 -> Launch instances**.
2. **Name:** VPC-Web-Server.
3. **AMI:** Amazon Linux 2023.
4. **Network Settings (Click Edit):**
 - **VPC:** Select CCL-Custom-VPC.
 - **Subnet:** Select a **Public Subnet** (important: must be Public!).
 - **Auto-assign public IP:** Select **Enable**.
 - **Security Group:** Select your existing Web-Server-SG.
5. **Advanced Details:** Scroll to the very bottom to the **User data** box and paste this:

us-east-1.console.aws.amazon.com

Search [Option+S]

United States (N. Virginia) prathamved (213348783129)

EC2 > Instances > Launch an instance

key pair name - required

Select Create new key pair

▼ Network settings Info

VPC - required Info

vpc-0fd836282f56562e (CCL-Custom-VPC-vpc) 10.0.0.0/16

Subnet Info

subnet-059d2ca242d9cda74 CCL-Custom-VPC-subnet-public1-us-east-1a VPC: vpc-0fd836282f56562e Owner: 213348783129 Availability Zone: us-east-1a (use1-az1) Zone type: Availability Zone IP addresses available: 4096 CIDR: 10.0.0.0/20

Create new subnet

Auto-assign public IP Info

Enable

Firewall (security groups) Info

A security group is a set of firewall rules that control the traffic for your instance. Add rules to allow specific traffic to reach your instance.

Create security group Select existing security group

Common security groups Info

Select security groups

Web-Server-SG sg-0f49e5bc65557aee3 VPC: vpc-0fd836282f56562e

Compare security group rules

Security groups that you add or remove here will be added to or removed from all your network interfaces.

► Advanced network configuration

▼ Summary

Number of instances Info

1

Software Image (AMI)

Amazon Linux 2023 AMI 2023.11...read more ami-0a59ec92177ec3fad

Virtual server type (instance type)

t3.micro

Firewall (security group)

Web-Server-SG

Storage (volumes)

1 volume(s) - 8 GiB

Cancel Launch instance Preview code

CloudShell Feedback Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

us-east-1.console.aws.amazon.com

Search [Option+S]

United States (N. Virginia) prathamved (213348783129)

EC2 > Instances > Launch an instance

Metadata response hop limit Info

2

Allow tags in metadata Info

Select

User data - optional Info

Upload a file with your user data or enter it in the field.

Choose file

```
#!/bin/bash
# Install and start Apache
yum update -y
yum install -y httpd
systemctl start httpd
systemctl enable httpd
# Create a custom landing page
echo "<h1>Success! Web Server launched in a Custom VPC</h1>" > /var/www/html/index.html
```

User data has already been base64 encoded

▼ Summary

Number of instances Info

1

Software Image (AMI)

Amazon Linux 2023 AMI 2023.11...read more ami-0a59ec92177ec3fad

Virtual server type (instance type)

t3.micro

Firewall (security group)

Web-Server-SG

Storage (volumes)

1 volume(s) - 8 GiB

Cancel Launch instance Preview code

CloudShell Feedback Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

Bash

```
#!/bin/bash
```

```
# Install and start Apache
```

```
yum update -y
```

```
yum install -y httpd
```

```
systemctl start httpd
```

```
systemctl enable httpd
```

```
# Create a custom landing page
```

```
echo "<h1>Success! Web Server launched in a Custom VPC</h1>" > /var/www/html/index.html
```

6. Click **Launch instance**.

Step 4: Verify Connectivity

1. Wait for the instance **Status Check** to pass.
2. Select the instance and copy the **Public IPv4 DNS** (e.g., ec2-xx-xx-xx.compute-1.amazonaws.com).
3. Paste that address into your browser.
4. If it loads, you have successfully configured:
 - **Internet Gateway:** Allowing traffic in.
 - **Route Tables:** Directing traffic to the public subnet.
 - **Subnets:** Separating public and private resources.



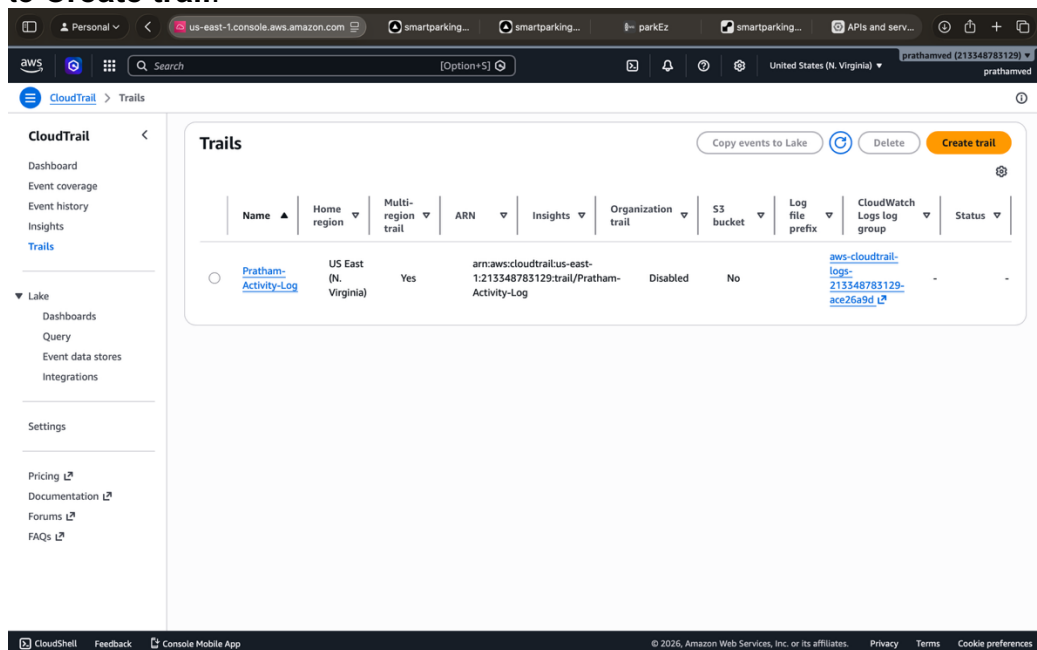
Q9

This is an essential task for **Auditing** and **Security**. CloudTrail acts like a "CCTV camera" for your AWS account, recording every click, command, and API call made by any user or service.

Step 1 & 2: Open CloudTrail and Create a Trail

1. Search for **CloudTrail** in the AWS Management Console.
2. Click **Create trail**.
3. **Trail name:** Pratham-Activity-Log.
4. **Storage location:** Select **Create new S3 bucket**.
5. **Trail Log Bucket and Folder:** AWS will suggest a unique name like aws-cloudtrail-logs-213348783129-xxxx. Leave it as is.
6. **Apply trail to all regions:** Ensure this is set to **Yes**. This ensures that even if someone launches an instance in London or Tokyo, it gets logged.

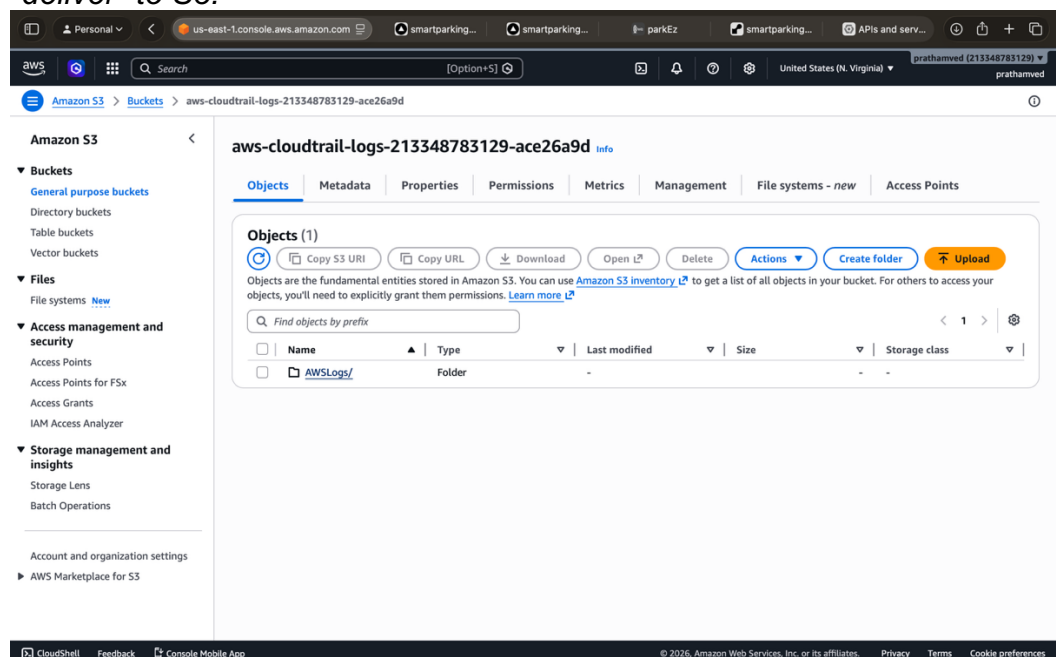
- Click **Next**, keep "Management events" checked, and click **Next** again to **Create trail**.



Step 3: The S3 Bucket (Verification)

AWS actually creates the S3 bucket for you during Step 2.

- Go to the **S3 Console**.
- Search for the bucket name you just saw in the CloudTrail setup.
- Open the bucket. You will see a folder structure like AWSLogs/AccountID/CloudTrail/.
 - Note: It may be empty for a few minutes; logs take a bit of time to "deliver" to S3.*



Step 4: Perform an AWS Action (The Trigger)

To test if it's working, we need to create a "paper trail":

1. Go to the **EC2 Dashboard**.
2. Select your **VPC-Web-Server** (or any instance).
3. Click **Instance state** -> **Stop instance**.
4. Once it stops, click **Instance state** -> **Start instance**.

Step 5: Verify the Log (The Evidence)

CloudTrail has two places to check: the **Event History** (fastest) and the **S3 Bucket** (permanent storage).

Checking Event History (Immediate):

1. Go back to the **CloudTrail Dashboard**.
2. Click **Event history** in the left menu.
3. Look for events like:
 - StopInstances
 - StartInstances
4. Click on the event name. You will see exactly **who** did it (your username), **when**, and from which **IP address**.

The screenshot shows the AWS CloudTrail Event History page. The left sidebar contains the CloudTrail navigation menu. The main content area displays a table of events. The table has the following columns: Event name, Event time, User name, Event source, Resource type, and Resource name. The events listed are:

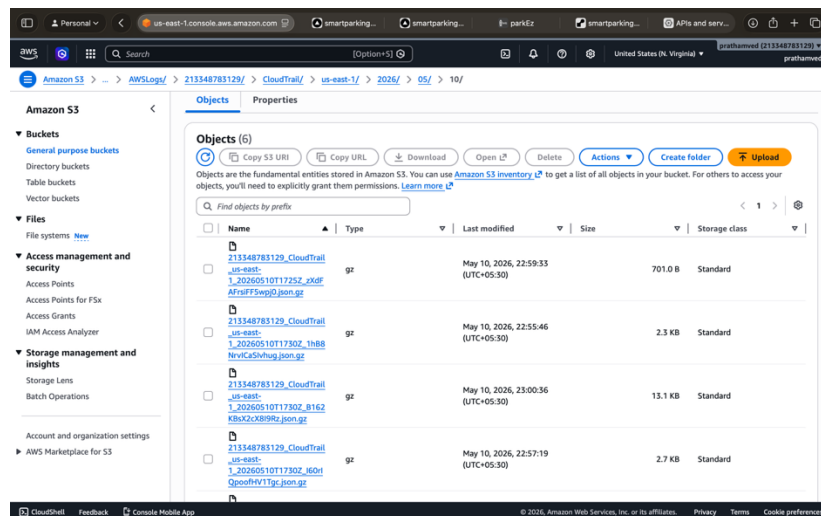
Event name	Event time	User name	Event source	Resource type	Resource name
StopInstances	May 10, 2026, 22:56:51 (UTC+0...)	root	ec2.amazonaws.com	AWS::EC2::Instance	i-06d9cb787f5
CreateLogStream	May 10, 2026, 22:55:15 (UTC+0...)	myStopinator	logs.amazonaws.com	-	-
PutEventSelectors	May 10, 2026, 22:54:07 (UTC+0...)	root	cloudtrail.amazonaws.com	AWS::CloudTrail::Trail	Pratham-Activi
StartLogging	May 10, 2026, 22:54:07 (UTC+0...)	root	cloudtrail.amazonaws.com	AWS::CloudTrail::Trail	arn:aws:cloudtr
CreateTrail	May 10, 2026, 22:54:06 (UTC+0...)	root	cloudtrail.amazonaws.com	AWS::CloudTrail::Trail	Pratham-Activi
PutBucketEncryption	May 10, 2026, 22:54:05 (UTC+0...)	root	s3.amazonaws.com	AWS::S3::Bucket	aws-cloudtrail-
PutBucketPolicy	May 10, 2026, 22:54:04 (UTC+0...)	root	s3.amazonaws.com	AWS::S3::Bucket	aws-cloudtrail-
CreateBucket	May 10, 2026, 22:54:03 (UTC+0...)	root	s3.amazonaws.com	AWS::S3::Bucket	aws-cloudtrail-
RegisterManagedInst...	May 10, 2026, 22:44:38 (UTC+0...)	i-060ef2e5e0087c...	ssm.amazonaws.com	-	-

0 / 5 events selected

Checking S3 (Formal Storage):

1. Go to your CloudTrail S3 bucket.
2. Navigate deep into the folders: Region > Year > Month > Day.

3. You will find .json.gz files. These are the raw logs containing the same data you saw in Event History.



Q9 This solution provides a step-by-step guide to implementing Infrastructure as Code (IaC) using Terraform to provision an AWS EC2 instance.

Step 1: Install and Configure Terraform

- **Install:** Download the Terraform binary for your operating system (Windows/Linux/macOS) from the official website and add it to your system's PATH.
- **Verify:** Open your terminal and run `terraform -v` to ensure it is installed correctly.
- **AWS CLI:** Install the AWS CLI and run `aws configure` to set your **Access Key**, **Secret Key**, and **Default Region** so Terraform can authenticate with your account.

Step 2: Set Up the Working Directory

- Create a new folder for your project (e.g., `ccl-terraform-lab`).
- Open your terminal/command prompt and navigate into this directory.

Step 3: Create the Terraform Configuration

Create a file named `main.tf` in your directory and add the following code to define the provider, security group, and EC2 instance:

Terraform

```
# 1. Configure the AWS Provider
provider "aws" {
  region = "us-east-1"
```

```
}
```

2. Create Security Group for SSH (22) and HTTP (80) [cite: 112]

```
resource "aws_security_group" "allow_web_ssh" {
  name      = "allow_web_ssh"
  description = "Allow SSH and HTTP inbound traffic"

  ingress {
    from_port = 22
    to_port   = 22
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"] # For SSH access [cite: 112]
  }

  ingress {
    from_port = 80
    to_port   = 80
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"] # For HTTP access [cite: 112]
  }

  egress {
    from_port = 0
    to_port   = 0
    protocol  = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}
```

3. Launch Amazon EC2 Instance [cite: 110]

```
resource "aws_instance" "my_ec2" {
  ami          = "ami-0c55b159cbf0" # Amazon Linux 2 AMI [cite: 111]
  instance_type = "t2.micro"         # Free Tier type [cite: 111]
  vpc_security_group_ids = [aws_security_group.allow_web_ssh.id]

  tags = {
    Name = "Terraform-Lab-Instance"
  }
}
```

Step 4: Initialize and Provision

1. **Initialize:** Run terraform init to download the necessary AWS provider plugins.
2. **Preview:** Run terraform plan to see exactly what resources Terraform will create before making changes.
3. **Apply:** Run terraform apply. Type yes when prompted to provision the resources on AWS.

Step 5: Verification and Validation

- **Console Check:** Log in to your AWS Management Console and navigate to **EC2 > Instances** to verify your instance is "Running".
- **SSH Access:** Use your terminal to connect to the instance: `ssh -i "your-key.pem" ec2-user@<public-ip>` to validate the deployment.

Step 6: Clean Up

- To avoid unwanted charges, run terraform destroy.
- Type yes to confirm the removal of the EC2 instance and the security group.

Q10

Complete Step-by-Step Guide for AWS Elastic Beanstalk (PaaS) Deployment

Follow these steps exactly to complete Task 11 of your practical exam.

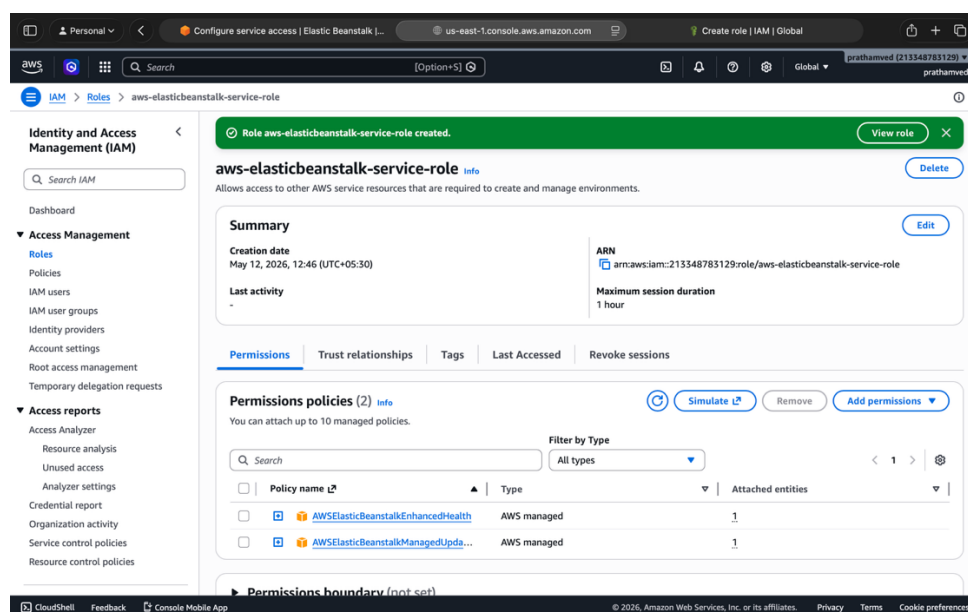
1. Application Initialization

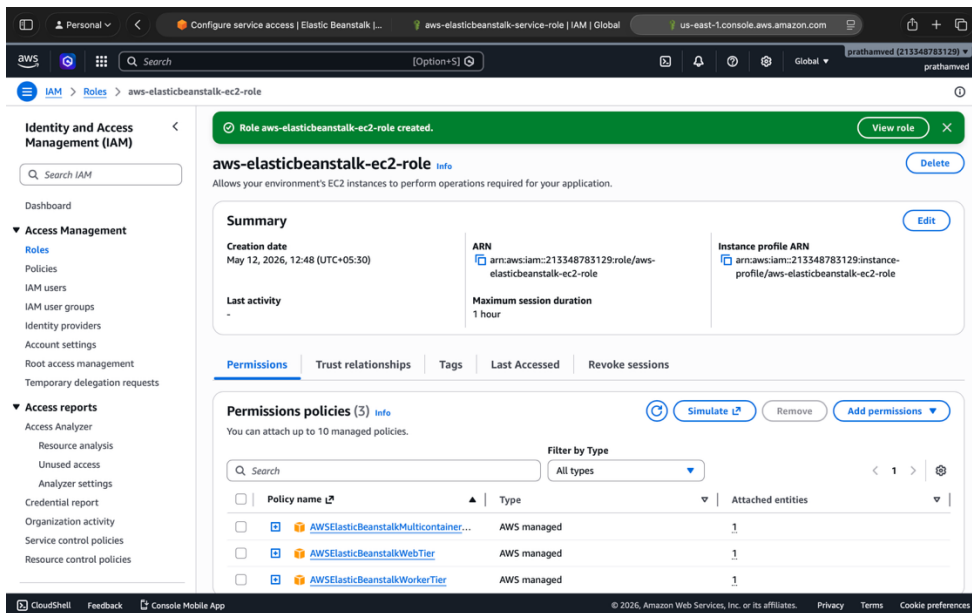
- **Open Console:** Navigate to **Elastic Beanstalk** in the AWS Management Console.
- **Create App:** Click **Create application**.
- **Name:** Enter CCL-PaaS-App.
- **Environment Tier:** Select **Web server environment**.
- **Platform:** Select **Python** (or Node.js/PHP) from the dropdown.
- **Application Code:** Select **Sample application**.
- **Configuration:** Choose **Single instance (free tier eligible)**.

2. Service Access (IAM Roles)

- **Service Role:** Select aws-elasticbeanstalk-service-role from the dropdown.
- **EC2 Instance Profile:** Select aws-elasticbeanstalk-ec2-role from the dropdown.

Note: If these are missing, create them in the IAM console with the permissions `AWSElasticBeanstalkWebTier` and `AWSElasticBeanstalkManagedUpdatesCustomerRolePolicy`.





- **EC2 Key Pair:** Select your key (e.g., cal).

3. Networking and Subnets

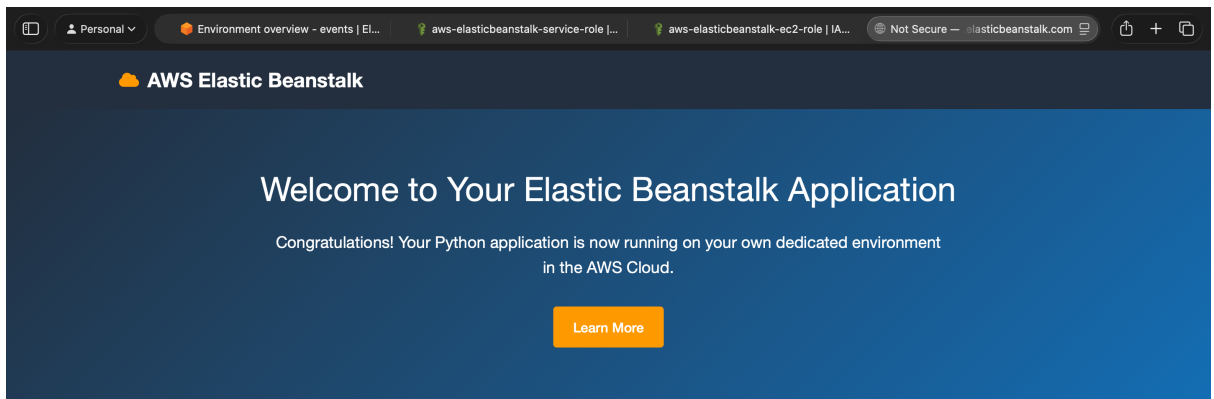
- **VPC:** Select the **Default VPC** (172.31.0.0/16).
- **Instance Subnets:** Check the boxes for **all** available Availability Zones (e.g., us-east-1a, us-east-1b) to ensure successful provisioning.
- **Public IP:** Ensure **Activated** is checked so the instance is accessible via the internet.

4. Instance and Scaling

- **Instance Type:** Verify it is set to t2.micro (within Free Tier limits).
- **Scaling:** Keep as **Single Instance** for the lab to avoid unnecessary resource creation.

5. Deployment and Verification

- **Review:** Click **Skip to Review** and then click **Submit**.
- **Wait:** The process takes 3–5 minutes. Monitor the events log for "Environment health is Green".
- **Access:** Click the **Domain URL** at the top of the dashboard to open your live sample website.



Benefits of AWS Elastic Beanstalk

Discover why thousands of developers rely on AWS Elastic Beanstalk to deploy and manage their applications.



Easy to Get Started

Upload your code and Elastic Beanstalk



Maximum Flexibility

With Elastic Beanstalk, you have the



Managed Platform

Elastic Beanstalk automatically manages

6. Post-Lab Management

- **Health Check:** Click the **Health** tab to show the examiner real-time status.
- **Cleanup:** Click **Actions > Terminate Environment** to delete all resources once you have finished your viva.

